Databases

Silviu Maniu
(original slides by M. Cortes-Cornax and M-C. Fauvet)

Univ. Grenoble Alpes – IM²AG / LIG

2025/2026

Chapter 1 – Organization

1. Organization
2. Course contents

Lecturer

Silviu Maniu	Professor of Computer Science, UGA, LIG

E-mail: `silviu.maniu@univ-grenoble-alpes.fr`

Moodle & Evaluation

Moodle: `https://im2ag-moodle.univ-grenoble-alpes.fr/course/view.php?id=676` (M1 AM IntroDB), password: **2025DBAM**

M1 AM	Project homeworks	$hw = (hw_1 + hw_2)/2$
	Final grade	$exam \times 0.6 + hw \times 0.4$

Notes:

- Project 1: modeling (UML)
- Project 2: relational + querying (SQL)
- Exam: everything, documents allowed

Chapter 1 – Organization

1. Organization
2. Course contents

Objectives

Understand the relational data model and SQL.
Grasp the basics of database design and implementation.

Ingredients:

- The relational data model
- The relational data language: SQL
- Conceptual modeling: UML
- Normalization, Transaction control, ...

Plan

- Part 1: relational data model and SQL
  1. Relational data model
  2. SQL – querying
  3. SQL – data definition, modification
- Part 2: modeling databases
  1. UML to relational
  2. Normalizing relations
  3. Transactions (?)

3 Overview of DBMSs

Databases basics

A database consists of real world information, organized so that it can be queried and modified (possibly deleted) later

A consistent collection of structured data

- which model (a part of) the real world
- designed to fulfill specific needs

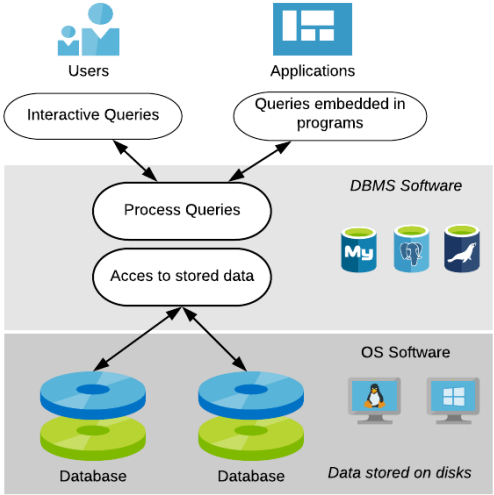
Examples:

- University students (first names, last names, addresses, student id, marks, ...)). Size: a few mega bytes
- World Data Centre for Climate: 220 terabytes[1] of web data, 6 petabytes[2] of additional data

[1] 1 tera= 10^{12}

[2] 1 peta= 10^{15}

A DataBase Management System (DBMS) is a set of software that provides means for storing and accessing data.



A Relational Database Management System (RDBMS)

A software system designed to support data-intensive applications

A Relational Database Management System (RDBMS)

A software system designed to support data-intensive applications

- with high-level access to the data (relational model, SQL)
- providing efficient storage and retrieval (disk/memory management)
- supporting multiple simultaneous users (privilege, protection)
- carrying out large numbers of operations (transaction management)
- maintaining reliable access to the stored data (backup, recovery)

DBMSs deal with many similar issues to operating systems.

DBMS Architecture

Three-layer architecture:

- **Presentation layer**: user interface, applications, Web pages, etc.
- **Logical layer**: relations/tables, queries (SQL), constraints, transactions – this is our focus
- **Physical layer**: storage on disk (files, indexes, etc.), caching, etc.

Main property: the functionality of a layer does not depend on the *implementation* of the other layers.

As a Summary

DBMSs provide access to valuable information resources in an environment that is:

- *Shared* - concurrent access by multiple users
- *Unstable* - potential for hardware/software failure

It involves techniques to ensure that each user sees the system as:

- *Unshared* - their work is not inadvertantly affected by other users
- *Stable* - the data survives in the face of system failures

Reading Material / References

- J. Widom, *Introduction to databases*, `https://www.edx.org/course/databases-5-sql`
- J.-D. Ullman, *Principles of Database and Knowledge-Base Systems (Vol. 1)*, Computer Science Press, 1990.
- R. Elmasri, S.B. Navathe *Conception et architecture des bases de données relationnelles*, 4e édition - Pearson Education France, 2004. Traduction française de *Fundamentals of Database Systems*, 4th Edition, Addison-Wesley Professional

4 Preliminaries

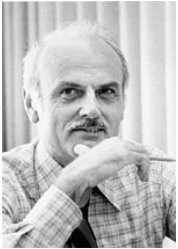
5 Definitions and Notations

6 Data design process

7 Specification of Relations: Example

The Relational Data Model...

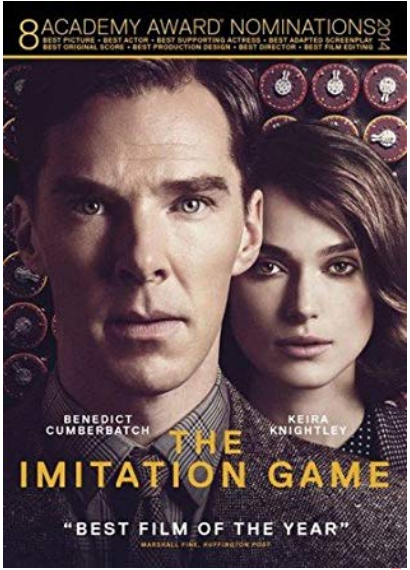
- was first introduced by Ted Codd in 1970,
- is attractive because of its simplicity and its mathematical foundations,
- uses the concept of mathematical relations as a means of modeling information,
- cannot be ignored because of its popularity.



source : `https://en.wikipedia.org/wiki/Edgar_F._Codd`

Turing Prize in 1981

Turing Prize in 1981

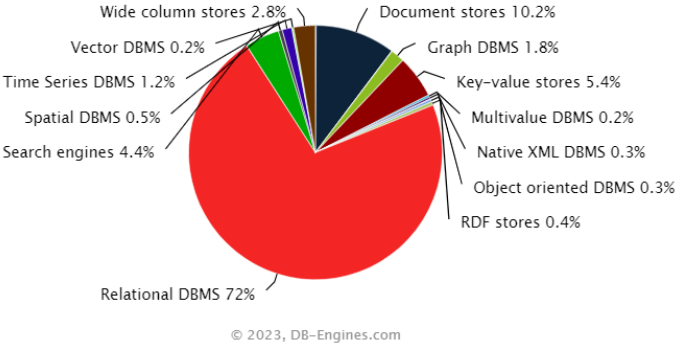


Michael Stonebraker (1943 -) - Turing Prize in 2014

Fundamental contributions for database management systems (DBMS). Inventor of PostgreSQL.

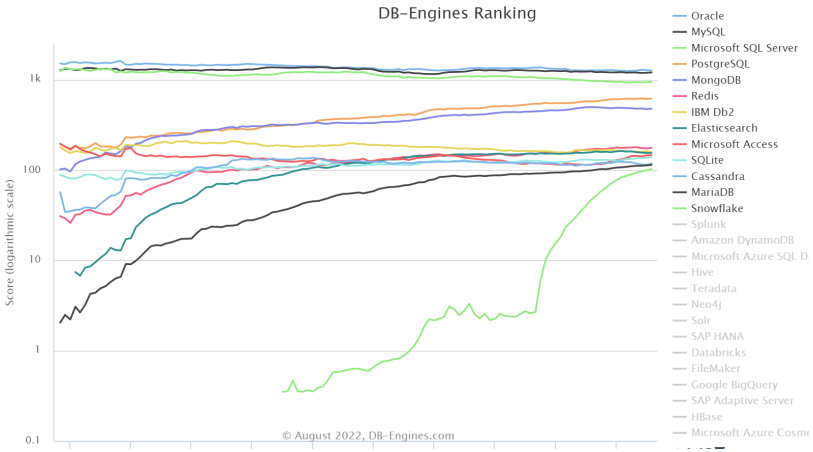
source : `https://en.wikipedia.org/wiki/Michael_Stonebraker`

Database Model Popularity



source : `https://db-engines.com/en/ranking_categories`

DB-Engines Ranking - Trend Popularity



source : `https://db-engines.com/en/ranking_trend`

How the Relational Model Represents Information?

Facts about a police registry:

- El Profesor, Tokio, Nairobi, Denver and Rio criminal's pseudonyms on the police registry,
- Tokio's real name is Silene, Nairobi's one is Agata, etc.
- Nairobi's speciality is Cunterfeiter, Tokio and Berlin are Robbers, etc.
- Rio's birthyear is 1995, Nairobi's one is 1987, etc.

pseudo	**name**	**specialty**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Concepts

Relational model:

1. A **database** is a **set of relations**
2. Every **relation** is supposed to have **semantically-coherent data**
3. **Relations are sets of tuples**: the **tuples** in a relation are **unique** (*set semantics*)
4. Every **column** or **attribute** in a relation represents a **property of the data**

Data manipulation:

1. **The results of querying and modifying relations are relations**

Operations on Relations

pseudo	name	specialty	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties
- **Selection** : Give criminals that born before 1985
- **Selection + projection**: what is the real name of Tokio?
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	specialty	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985
- **Selection + projection**: what is the real name of Tokio?
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	specialty	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985 $\longrightarrow$ {<El Profesor, Sergio, Band Leader, 1983>, <Berlin, Andres, Robber, 1979>}
- **Selection + projection**: what is the real name of Tokio?
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	specialty	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985 $\longrightarrow$ {<El Profesor, Sergio, Band Leader, 1983>, <Berlin, Andres, Robber, 1979>}
- **Selection + projection**: what is the real name of Tokio? $\longrightarrow$ {<Silene>}
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	specialty	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985 $\longrightarrow$ {<El Profesor, Sergio, Band Leader, 1983>, <Berlin, Andres, Robber, 1979>}
- **Selection + projection**: what is the real name of Tokio? $\longrightarrow$ {<Silene>}
- **Aggregation**: how many criminals are Robbers? $\longrightarrow$ {<2>}

Updating relations

- Nairobi's speciality has been changed to Robber

pseudo	**name**	**specialty**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	*Robber*	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Updating relations

- Nairobi's speciality has been changed to Robber
- Moscu is now identified in the police's registry. He's name is Agustin, he is a Burglar and his birthyear is 1968

pseudo	**name**	**specialty**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	*Robber*	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979
Moscu	*Agustin*	*Burglar*	*1968*

4 Preliminaries

5 Definitions and Notations

6 Data design process

7 Specification of Relations: Example

Set: Definition and Some Operations

Set semantics of databases: relations consist of **sets of tuples**.

- A set is a collection of elements (enclosed by braces) pairwise different and usually related to a particular domain.
  A = {'Robber', 'Hacker', 'Burglar'}
  F = {p ∈ Persons: Gender(p) = 'female'}
  G = {1, 5, 6, 3}

A and G are given by extension, while F is so by intension.

Set: Definition and Some Operations

Set semantics of databases: relations consist of **sets of tuples**.

- A set is a collection of elements (enclosed by braces) pairwise different and usually related to a particular domain.
  $A = \{\text{'Robber'}, \text{'Hacker'}, \text{'Burglar'}\}$
  $F = \{p \in Persons: Gender(p) = \text{'female'}\}$
  $G = \{1, 5, 6, 3\}$
- Membership:
  $3 \in \{1, 3, 5, 6\}$
  $8 \notin \{1, 3, 5, 6\}$

A and G are given by extension, while F is so by intension.

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$
- Union:
  $\{1, 3, 5, 6\} \cup \{10, 5, 3, 9\} = \{1, 5, 3, 6, 10, 9\}$

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$
- Union:
  $\{1, 3, 5, 6\} \cup \{10, 5, 3, 9\} = \{1, 5, 3, 6, 10, 9\}$
- Minus (asymetrical):
  $\{1, 3, 5, 6\} - \{10, 5, 3, 9\} = \{1, 6\}$
  $\{10, 5, 3, 9\} - \{1, 3, 5, 6\} = \{10, 9\}$

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$
- Union:
  $\{1, 3, 5, 6\} \cup \{10, 5, 3, 9\} = \{1, 5, 3, 6, 10, 9\}$
- Minus (asymetrical):
  $\{1, 3, 5, 6\} - \{10, 5, 3, 9\} = \{1, 6\}$
  $\{10, 5, 3, 9\} - \{1, 3, 5, 6\} = \{10, 9\}$
- Inclusion:
  $\{\ \} \subseteq \{10, 5, 3, 9\}$ ($\{\ \}$ is also denoted $\emptyset$)
  $\{9, 10\} \subseteq \{10, 5, 3, 9\}$
  $\{1, 9, 10, 3, 5, 7\} \not\subseteq \{10, 5, 3, 9\}$

Domain, Relation, Attribute, Schema

- A domain is a set of atomic values (strings, s,...).
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'}, {integers > 1900}

Domain, Relation, Attribute, Schema

- A domain is a set of atomic values (strings, s,...).
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'}, {integers > 1900}
- A relation (table) is a subset of the cartesian product of a set of domains.
  {<'El Profesor',1983>, <'Tokio',1989>, <'Nairobi',1987>, <'Denver',1987>, <'Rio',1979>}
  $\subseteq$
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'} X {integers > 1900}

Domain, Relation, Attribute, Schema

- A domain is a set of atomic values (strings, s,...).
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'}, {integers > 1900}
- A relation (table) is a subset of the cartesian product of a set of domains.
  {<'El Profesor',1983>, <'Tokio',1989>, <'Nairobi',1987>, <'Denver',1987>, <'Rio',1979>}
  $\subseteq$
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'} X {integers > 1900}
- An attribute specifies a name for a domain in a relation.
  dom(birthyear) = { integer > 1900}

Domain, Relation, Attribute, Schema

- A domain is a set of atomic values (strings, s,...).
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'}, {integers > 1900}
- A relation (table) is a subset of the cartesian product of a set of domains.
  {<'El Profesor',1983>, <'Tokio',1989>, <'Nairobi',1987>, <'Denver',1987>, <'Rio',1979>}
  $\subseteq$
  {'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'} X {integers > 1900}
- An attribute specifies a name for a domain in a relation.
  dom(birthyear) = { integer > 1900}
- A relation schema is specified by its name and a set of attributes.
  Criminals (pseudo, name, speciality, birthyear, crimeId)

Example:

Attributes

Relation Name

Criminals	pseudo	name	speciality	birthyear	crimeId
	El Profesor	Sergio	Band Leader	1983	2017052
	Tokio	Silene	Robber	1989	2017052
	Nairobi	Agata	Counterfeiter	1987	2017052
	Denver	Daniel	Burglar	1987	2017052
	Rio	Anibal	Hacker	1995	2017052
	Berlin	Andres	Robber	1979	2017052

Relation Schema

Tuples / n-uplets

Relation Instance

Attributes and tuples have no order

Interpretation

An interpretation of a relation schema as a predicate:

Criminals (pseudo, name, speciality, birthyear, crimeId)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s, birthyear b and has been involved in crime crimeId cn. */*

This is useful to understand a relation schema.

Relational Constraints

- *Domain constraint:*
  domain(A)=T specifies that the values of A must be from the type T.

Relational Constraints

- *Domain constraint:*
  domain(A)=T specifies that the values of A must be from the type T.
- *Key constraint:*
  $\underline{X}$ specifies an uniqueness constraint so that tuples of the relation are pairwise different for X (X set of attributes).

Relational Constraints

- *Domain constraint:*
  domain(A)=T specifies that the values of A must be from the type T.
- *Key constraint:*
  $\underline{X}$ specifies an uniqueness constraint so that tuples of the relation are pairwise different for X (X set of attributes).
- *Referential integrity constraint:*
  R on attributes X refers to S on attributes Y (Notation: $R[X] \subseteq S[Y]$). All tuples in R restricted to X must have a corresponding tuple in S that matches on Y.

Relational Constraints: Keys

Key constraint:
<u>X</u> specifies an uniqueness constraint so that tuples of the relation are pairwise different for X (X set of attributes).

pseudo	**name**	**specialty**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Relational Constraints: Referential Integrity

Referential integrity constraint:
R on attributes X refers to S on attributes Y (Notation: $R[X] \subseteq S[Y]$). All tuples in R restricted to X must have a corresponding tuple in S that matches on Y.

pseudo	**name**	**specialty**	**birthyear**	**crimeId**
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

crimeId	**type**	**location**
2017052	Heist	FNMT

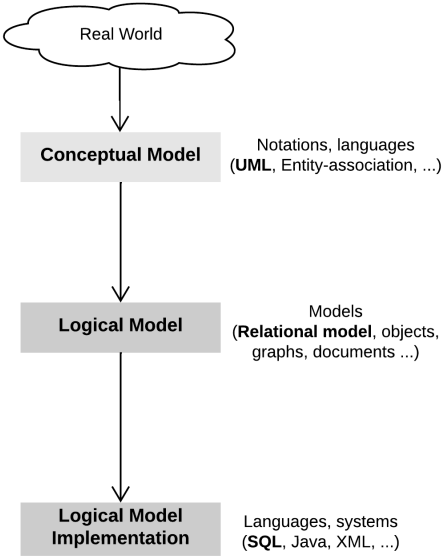
4 Preliminaries

5 Definitions and Notations

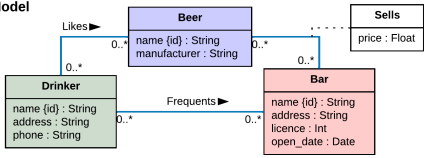
6 Data design process

7 Specification of Relations: Example

Data design process



Conceptual Model



Logical Model

Beers (beer_name, beer_manufacturer)

Drinkers (drinker_name, drinker_address, drinker_phone)

Bars (bar_name, bar_address, bar_licence, bar_open_date)

Likes (drinker_name, beer_name)

Likes[drinker_name] ⊆ Drinkers[drinker_name]

Likes[beer_name] ⊆ Beers[beer_name]

Frequents (drinker_name, bar_name)

Frequents[drinker_name] ⊆ Drinkers[drinker_name]

Frequents[bar_name] ⊆ Bars[bar_name]

Sells (bar_name, beer_name, sells_price)

sells_price > 0

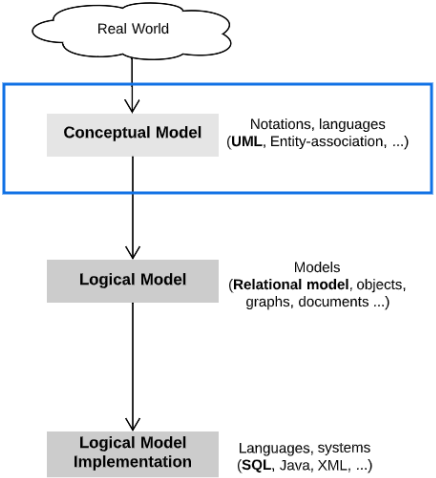
Sells[beer_name] ⊆ Beers[beer_name]

Sells[bar_name] ⊆ Bars[bar_name]

Logical Model Implementation

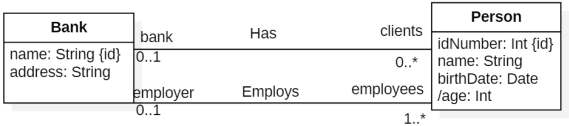
```
-- SQLite Syntax
CREATE TABLE Beers (
  beer_name TEXT NOT NULL,
  -- NOT NULL not necessary for PK attributes in some systems such as ORACLE
  beer_manufacturer TEXT,
  CONSTRAINT pk_beers_c0 PRIMARY KEY (beer_name)
);

CREATE TABLE Bars (
      bar_name TEXT NOT NULL,
      bar_address TEXT,
      bar_licence DATE,
      bar_open_date DATE,
      CONSTRAINT pk_bars_c0 PRIMARY KEY (bar_name)
);
```

Data design process



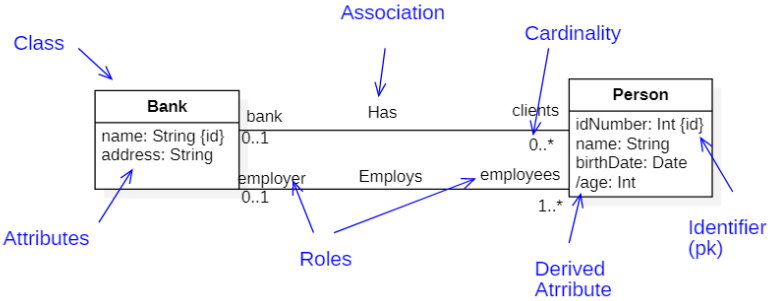
The Main UML Class Diagram* Elements

*UML Adaptation for DBs

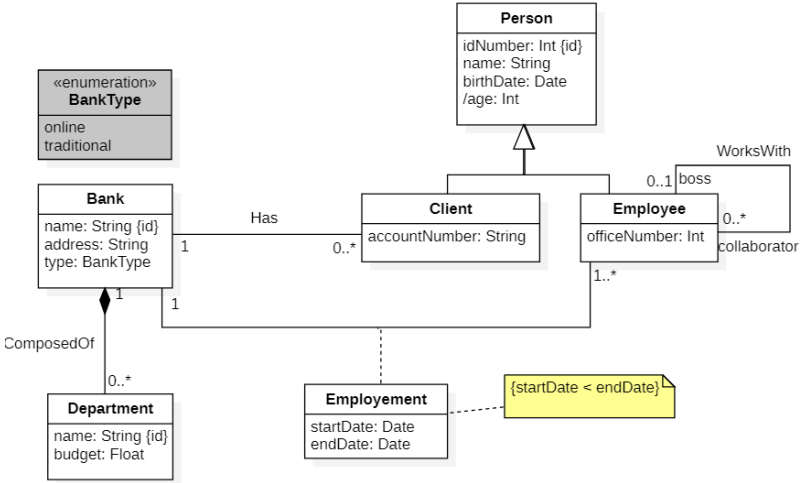


The Main UML Class Diagram* Elements

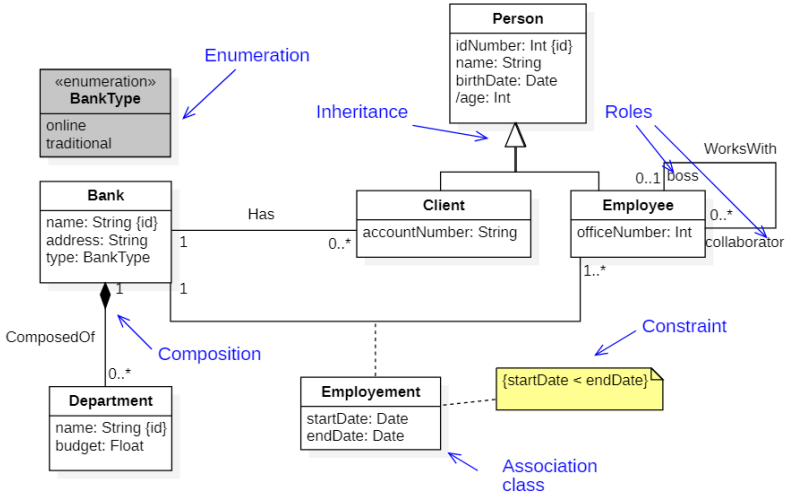
*UML Adaptation for DBs



Extended UML class diagram



Extended UML class diagram

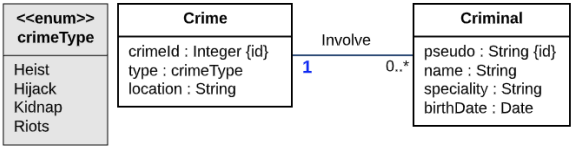


4 Preliminaries

5 Definitions and Notations

6 Data design process

7 Specification of Relations: Example



Criminals (pseudo, name, speciality, birthyear, crimeId)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s, a birthyear b and she(he) has been involved in the crime crimeId cn. */*

Crimes(crimeId, type, location)

/ <n, t, l> ∈ Crimes ⟺ the crime identified by a crimeId n, has a type t (Heist, Hijack, Riots, Kidnap) and a location l */*

Domain constraints:

domain (pseudo) = domain (name) = domain (speciality) = domain (location) = strings

domain (birthyear) = number > 1900

domain (crimeId) = number > 0

domain (type) = {'Heist', 'Hijack', 'Riots', 'Kidnap'}

Referential integrity constraint:

Criminals[crimeId] ⊆ Crimes[crimeId]

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

crimeId	type	location
2017052	Heist	FNMT

8 Introduction

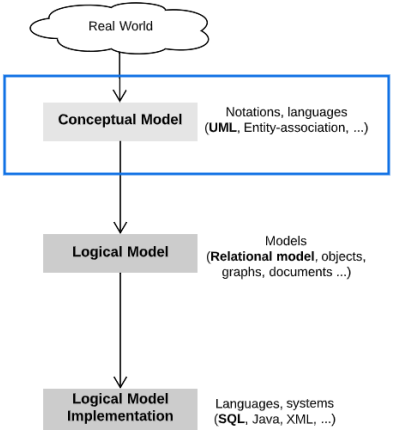
9 Mapping classes

10 Mapping Associations and Association classes

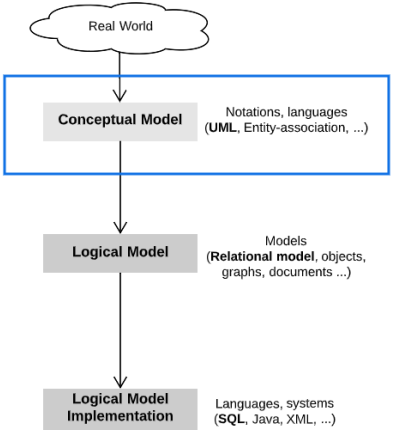
11 Mapping class Hierarchy

12 Concluding Remarks

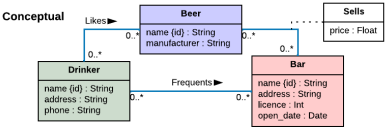
Data design process



Data design process



Conceptual



Logical

Beers (beer_name, beer_manufacturer)

Drinkers (drinker_name, drinker_address, drinker_phone)

Bars (bar_name, bar_address, bar_licence, bar_open_date)

Likes (drinker_name, beer_name)

Likes[drinker_name] ⊆ Drinkers[drinker_name]

Likes[beer_name] ⊆ Beers[beer_name]

Frequents (drinker_name, bar_name)

Frequents[drinker_name] ⊆ Drinkers[drinker_name]

Frequents[bar_name] ⊆ Bars[bar_name]

Sells (bar_name, beer_name, sells_price)

sells_price > 0

Sells[beer_name] ⊆ Beers[beer_name]

Sells[bar_name] ⊆ Bars[bar_name]

SQL Implementation

```
-- SQLite Syntax
CREATE TABLE Beers (
 beer_name TEXT NOT NULL,
 -- NOT NULL not necessary for PK attributes in some systems such as ORACLE
 beer_manufacturer TEXT,
 CONSTRAINT pk_beers_c0 PRIMARY KEY (beer_name)
);

CREATE TABLE Bars (
     bar_name TEXT NOT NULL,
     bar_address TEXT,
     bar_licence DATE,
     bar_open_date DATE,
     CONSTRAINT pk_bars_c0 PRIMARY KEY (bar_name)
);

CREATE TABLE Sells(
 bar_name TEXT NOT NULL,
 beer_name TEXT NOT NULL,
 sells_price REAL NOT NULL,
 CONSTRAINT pk_sells_c0 PRIMARY KEY (bar_name,beer_name),
```

Relational vs. UML

The relational model and UML class diagram have some obvious correspondences:

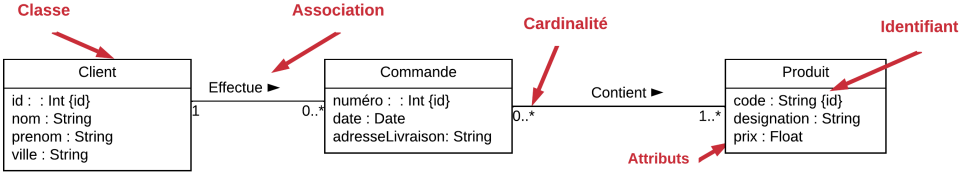
Relational Model	*UML class Diagram*
Attributes	Attributes
Relation schema	classes and Associations
Tuple	Object value

The major difference:

1. Relations are used to model both classes and some associations

Example UML to Relational Model (1)

UML (Conceptual level):

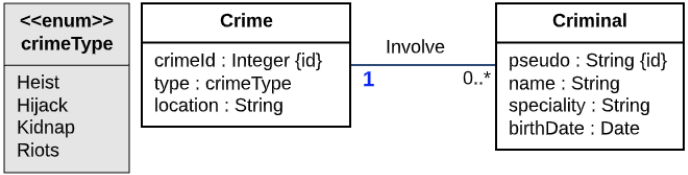


Relational model (logical level):

Clients(<u>id_client</u>, nom_client, prenom_client, ville_client)
Commandes(<u>numero_commande</u>, date_commande, adresse_livraison_commande, id_client)
Produits(<u>code_produit</u>, designation_produit, prix_produit)
ProduitsCommandes(<u>numero_commande, code_produit</u>)/* *suite à l'asso many-to-many Contient* */

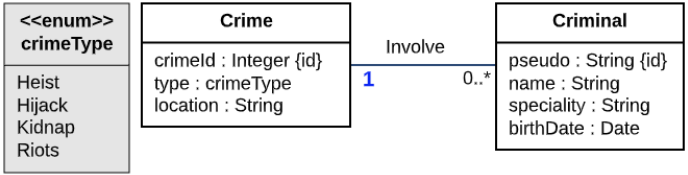
Commandes[id_client] $\subseteq$ Clients[id_client]
ProduitsCommandes[numero_commande] $=$ Commandes[numero_commande]
/* *égalité car toute commande apparaît dans la table ProduitsCommandes (1..*)* */
ProduitsCommandes[code_produit] $\subseteq$ Produits[code_produit]

Example UML to Relational Model (2)



Relational model:

Example UML to Relational Model (2)



Relational model:

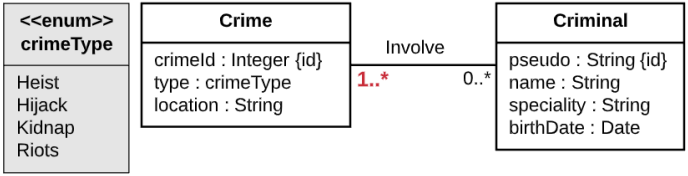
Criminals (pseudo, name, speciality, birth_date, crimeId)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s, a birth_year b and she(he) has been involved in the crime crimeId cn. */*

Crimes(crimeId, type, location)

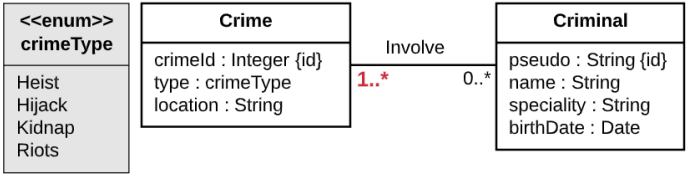
/ <n, t, l> ∈ Crimes ⟺ the crime identified by a crimeId n, has a type t (Heist, Hijack, Riots, Kidnap) and a location l */*

Example UML to Relational Model (3)



Relational model:

Example UML to Relational Model (3)



Relational model:

Criminals (pseudo, name, speciality, birth_year)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s and a birth_year b. */*

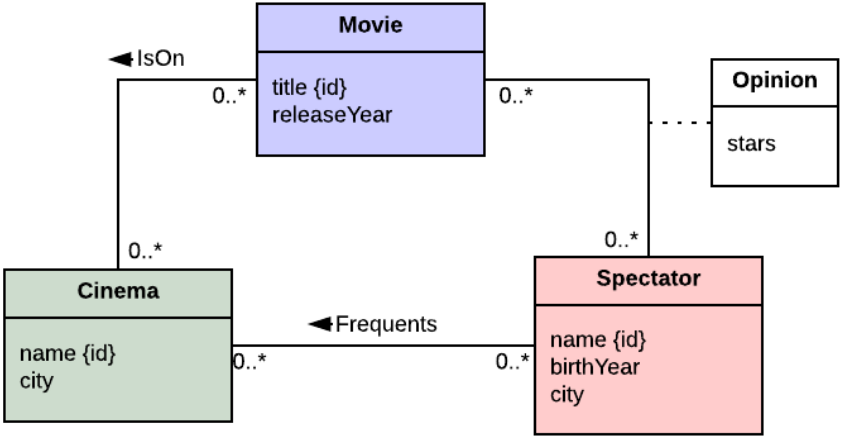
Crimes(crimeId, type, location)

...

Involvements(crimeId, pseudo)

/ <ci, cp> ∈ Involvements ⟺ the crime identified by a crime Id ci involves criminal identified by pseudo cp */*

Example UML to Relational Model (4)



Example UML to Relational Model (4)

Toy Database Relational Model

- *Movies (title, release_year)*
  $< t, y > \in$ ***Movies*** $\iff$ {the movie is identified by the title ***t*** and has a release year ***y***}
- *Cinemas (name, city)*
  $< n, c > \in$ ***Cinemas*** $\iff$ {the cinema identified by its name ***n*** is located in the city ***c***}
- *Spectators (name, birth_year, city)*
  $< n, b, c > \in$ ***Spectators*** $\iff$ ...
  Domains need to be detailed.
- *Frequents (spectator, cinema)*
  $< s, c > \in$ ***Frequents*** $\iff$ ...
  Frequents[spectator] $\subseteq$ Spectators[name]
  Frequents[cinema] $\subseteq$ Cinemas[name]
- *IsOn (movie, cinema)*
  $< m, c > \in$ ***IsOn*** $\iff$ ...
  IsOn[movie] $\subseteq$ Movies[title]
  IsOn[cinema] $\subseteq$ Cinemas[name]
- *Opinions (spectator, movie, stars)*
  $< sp, m, st > \in$ ***Opinions*** $\iff$ ...
  domain(stars) = {0, 1, 2, 3, 4, 5} Opinions[spectator] $\subseteq$ Spectators[name]
  Opinions[movie] $\subseteq$ Movies[title]

UML to Relational Mapping: Aims

Implementing a DBMS-based application needs to transform conceptual design into relational model.
Main objectives are:

- Represent all information contained in a class diagram (classes, associations and constraints)
- Avoid redundancy (produce a good-quality[3] relational schema)
- Provide a relational schema as simple as possible (avoid too many relations)

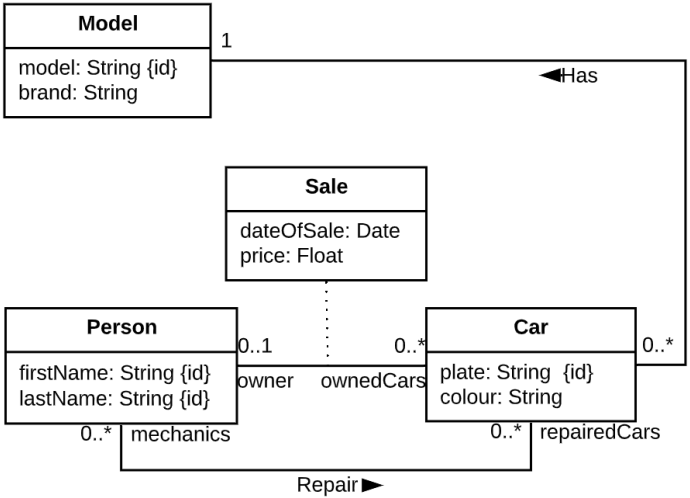
[3] 3NF schema (see later)

The Main UML Elements: Exercise

A garage:

- A car is identified by its number plate
- A car has a color, and it is always attached to a model and a brand. A model is unique and determines the brand.
- Different cars may be attached to the same brand and the same model (e.g., we may have 10 Renault Megane with different plates and colors)
- A person is identified by a first name and a last name.
- A car is owned by only one person (the owner) after a sale. A person can own several cars.
- The sale is captured by a timestamped snapshot as well as the price.
- A car can be repaired by several persons (mechanics), and a mechanic can repair several cars.

Corresponding UML class diagram



8 Introduction

9 Mapping classes

10 Mapping Associations and Association classes

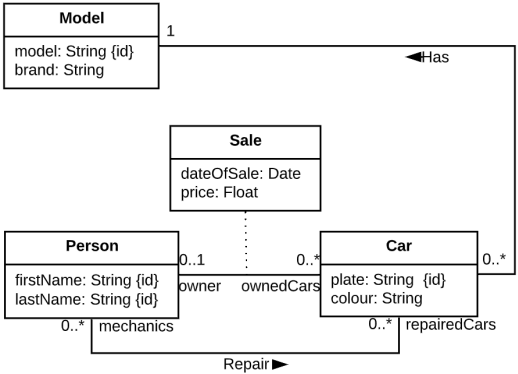
11 Mapping class Hierarchy

12 Concluding Remarks

Mapping classes

For classes that are not association class:

- One relation for each class
- Relation attributes are class attributes
- Id elements are part of a composed key



Models(model, brand, ...
Cars(plate, colour, ...
Persons(first_name, last_name, ...

8 Introduction

9 Mapping classes

10 Mapping Associations and Association classes

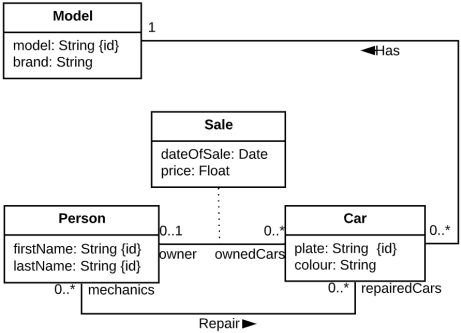
11 Mapping class Hierarchy

12 Concluding Remarks

Mapping Associations (1 to Many)

For associations 1 to Many (*):

- Add attributes in the * class referencing the key of the 1 role class
- As cardinality is 1, the new attribute (e.g., fk) cannot be null

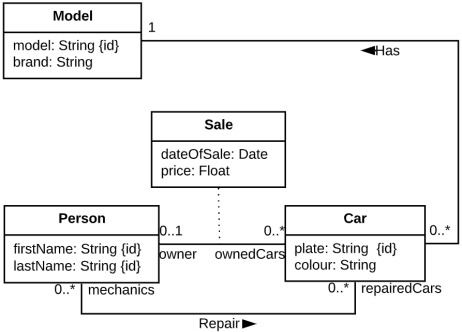


Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, **model**)
Persons (<u>first_name, last_name</u>)
...
Cars[model] $\subseteq$ Models[model]
model not null in Cars

Mapping Associations (0..1 to Many) Opt. 1 - Accept NULL values

For associations 0..1 to Many (*), first option :

- Add attributes in the * class referencing the key of the 0..1 role class (admit NULL values)



Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, model, **owner_first_name, owner_last_name, date_of_sale, price**)
Persons (<u>first_name, last_name</u>)
...

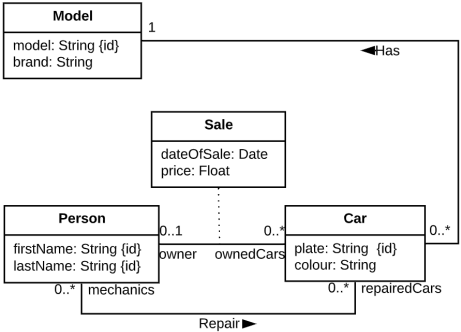
Cars[owner_first_name, owner_last_name] $\subseteq$ Persons[first_name, last_name]

owner_first_name, owner_last_name, date_of_sale, price **can be NULL**

Mapping Associations (0..1 to Many) Opt. 2 - Do not accept NULL values

For associations 0..1 to Many (*) second option :

- A new relation where the key of the 0..* role class is the key of the created relation.



Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, model)
Sales(owner_first_name, owner_last_name, <u>plate</u>, date_of_sale, price)
Persons (<u>first_name, last_name</u>)
...

$$\text{Sales[plate]} \subseteq \text{Cars[plate]}$$

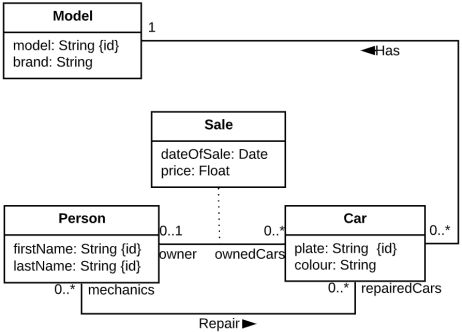
$$\text{Sales[owner_first_name, owner_last_name]} \subseteq \text{Persons[first_name,last_name]};$$

owner_first_name, owner_last_name, plate, date_of_sale, price **cannot be NULL**;

Mapping Associations (Many to Many)

For associations Many to Many:

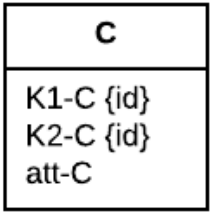
- A new relation represents the many to many association
- The key of the new relation is composed of attributes that reference the keys of the related relations



Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, model)
Sales (owner_first_name, owner_last_name, <u>plate</u>, date_of_sale, price)
Persons (<u>first_name, last_name</u>)
Reparations(<u>mech_first_name, mech_last_name, plate</u>)

Reparations[mech_first_name, mech_last_name] $\subseteq$ Persons[first_name, last_name];
Reparations[plate] $\subseteq$ Cars[plate];

Question 1 : UML class to Relational

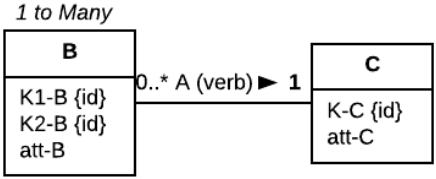


A. C(K1-C, K2-C, att-C)

B. C(K1-C, K2-C, att-C)

C. C(K1-C, K2-C, att-C)

Question 2 : UML 1 to Many association to Relational (class B)

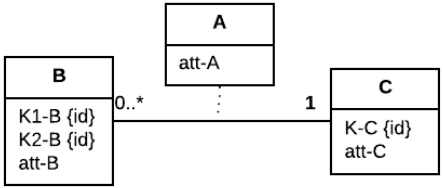


A. B(K1-B, K2-B, att-B)

B. B(K1-B, K2-B, att-B, K-C)

C. B(K1-B, K2-B, K-C, att-B)

Question 3 : UML Association class in 1 to Many association to Relational (class A and B)

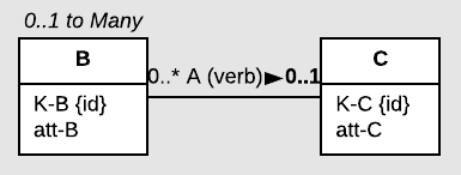


A. B(K1-B, K2-B, att-B, K-C, att-A)

B. B(K1-B, K2-B, att-B, K-C); A(att-A)

C. B(K1-B, K2-B, att-B); A(K1-B, K2-B, K-C, att-A)

Question 4 : UML 1..0 to Many Association to Relational (class B and Association A)



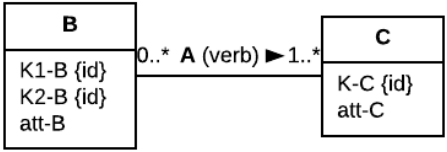
A. B(K-B, att-B); A(K-B, K-C)

B. B(K-B, att-B); A(K-B, K-C)

C. B(K-B, att-B, K-C)

Question 5 : UML Many to Many Association to Relational

Many to Many



A. B(<u>K1-B, K2-B</u>, att-B, <u>K-C</u>, att-C)

B. B(<u>K1-B, K2-B</u>, att-B); A(<u>K1-B, K2-B</u>,K-C); C(<u>K-C</u>, att-C)

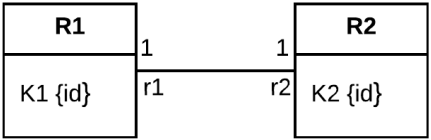
C. B(<u>K1-B, K2-B</u>, att-B); A(<u>K1-B, K2-B, K-C</u>); C(<u>K-C</u>, att-C)

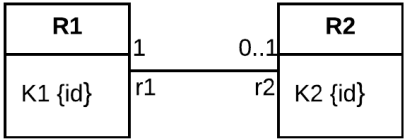
Reminder of Rules with Many to Many Asssocations

A new relation R is created (related to the association itself):

- K_1 candidate key in R_1, K_2 candidate key in R_2
- Attributes in R: $K_1 \cup K_2$
- Candidate key in R: $K_1 \cup K_2$
- Referential integrity constraints:
  $R[K_1] \subseteq R_1[K_1]$ and $R[K_2] \subseteq R_2[K_2]$

Mapping associations (0..1 or 1 on both roles)





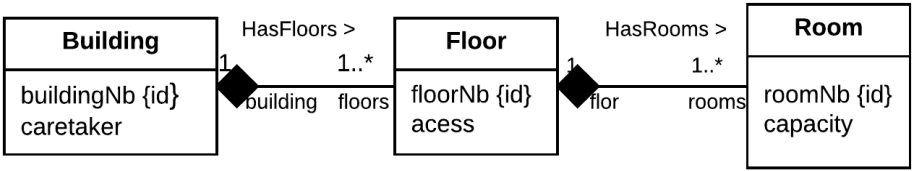
- 1—— 1. 2 options :
  1. K1 are attributes in R2 and R2[K1] = R1[K1],
     K1 is *another* candidate key in R2.
  2. K2 are attributes in R1 and R1[K2] = R2[K2],
     K2 is *another* candidate key in R1
- 1 —— 0..1. To avoid missing values, one option only:
  1. K1 are attributes in R2 and R2[K1] $\subseteq$ R1[K1],
     K1 is *another* candidate key in R2.

Association with Composition

We consider buildings located on a campus:

- Buildings have a caretaker and they are identified by their number.
- Building floors have specific access means, and in a given building, floors are identified by their number.
- Rooms have a fixed capacity, and in a given floor they are identified by their number.

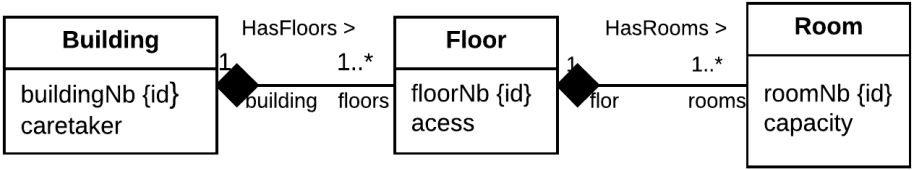
1. what are the **classes** involved?;
2. what are the **associations** involved ? (rel. name?, composition? multiplicity?, roles?)
3. what are the **identifiers** of the classes?



Association with Composition

For association with composition:

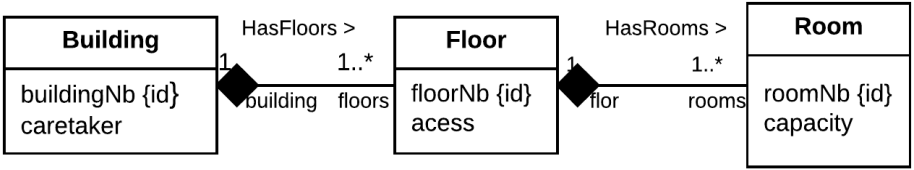
- The key of the 'whole' is included the key of the 'part'



Association with Composition

For association with composition:

- The key of the 'whole' is included the key of the 'part'



The relation schema is:

Buildings (buildingNb, caretaker)
Floors (floorNb, buildingNb, access)
Rooms (roomNb, floorNb, buildingNb, capacity)
Referential integrity constraints:
Floors[buildingNb] = Buildings[buildingNb]
Rooms[floorNb, buildingNb] = Floors[floorNb, buildingNb]

Not null contraints are implicitly associated with keys.

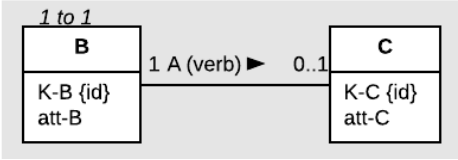
Question 6 : UML Composition to Relational (class B)

A. B(<u>K-B</u>, att-B)

B. B(<u>K-B</u>, <u>K-A</u>, att-B)

C. B(<u>K-B</u>, <u>K-A</u>, att-B)

Question 7 : UML 1 to 1 Association to Relational



A. B(<u>K-B</u>, att-B); C(<u>K-C</u>, att-C, <u>K-B</u>)

B. B(<u>K-B</u>, att-B); C(<u>K-C</u>, att-C, K-B)

C. B(<u>K-B</u>, att-B); A(<u>K-B,K-C</u>); C(<u>K-C</u>, att-C)

8 Introduction

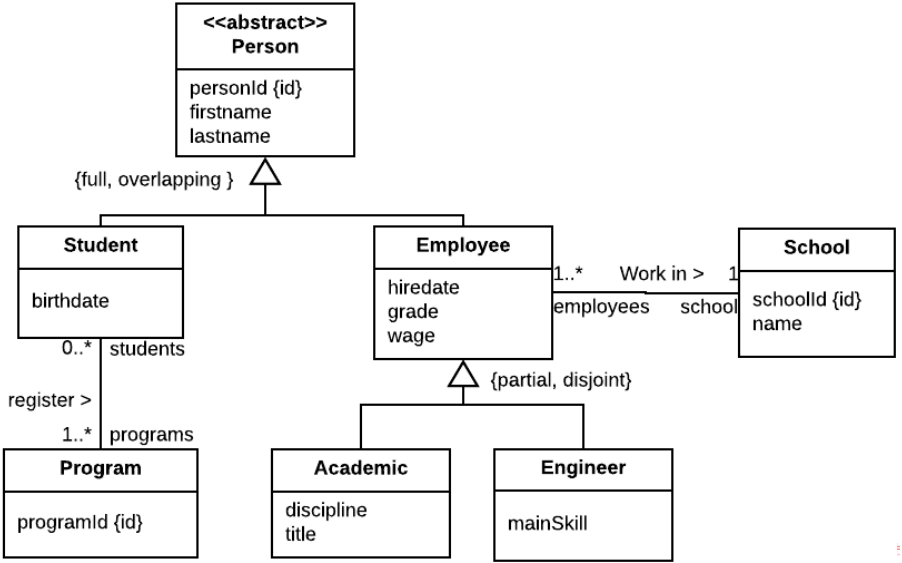
9 Mapping classes

10 Mapping Associations and Association classes

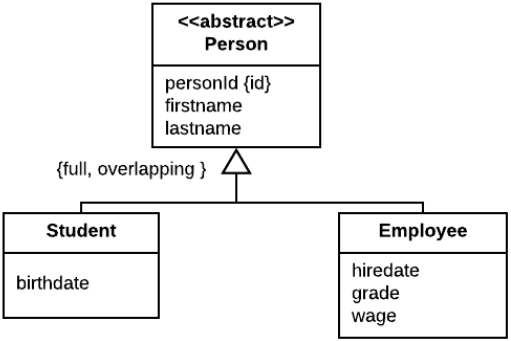
11 Mapping class Hierarchy

12 Concluding Remarks

Example: About a University

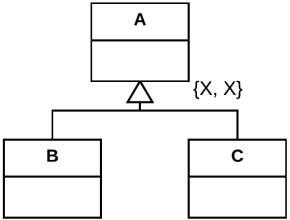


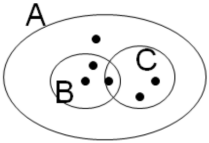
Example: About a University



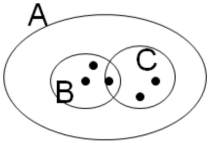
- An employee is a person (with a first_name and last_name) who has a hiredate, a grade and a wage
- A student is a person who has a birthdate
- A person may only be an employee or a student (full)
- A person can be an employee, a student or both (overlapping)

Inheritance constraints

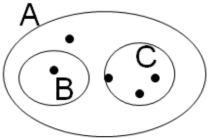




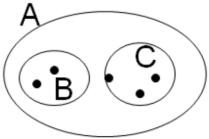
{partial, overlapping}



{full, overlapping}



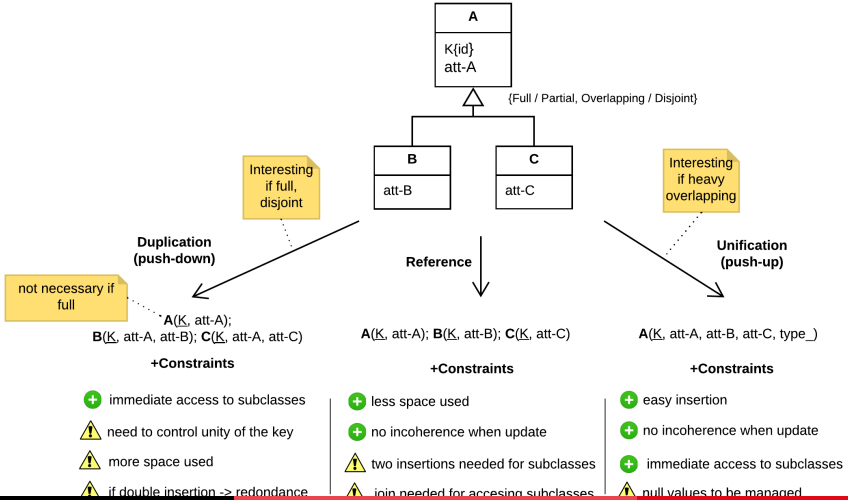
{partial, disjoint}



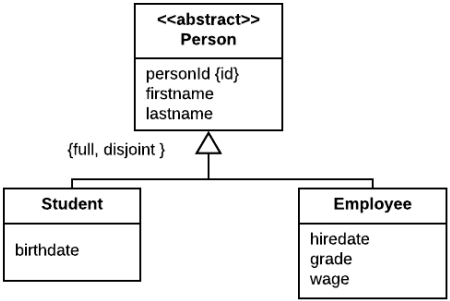
{full, disjoint}

Inheritance Mapping : three approaches

Usually, inheritance constraints may guide the used approach



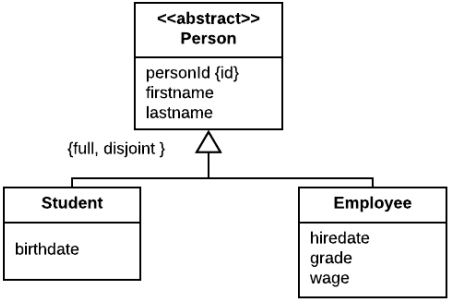
The University, Persons (Full, Disjoint Constraints)



- Given a person p: p is either a student or an employee (not both)
- Due to "full" constraint, there is no need for a relation Persons (there is no person that is neither student nor employee)
  1. Put Person properties in both relations Students and Employees (Duplicatation - Push down)

The University, Persons, Duplication Version

Relations related to sub-classes of Person all contain properties inherited from Persons:

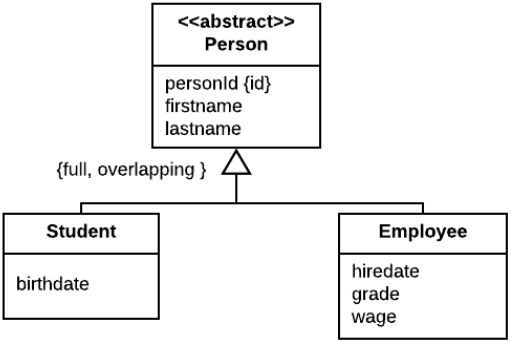


Students (personId, first_name, last_name, birthdate)
Employees (personId, first_name, last_name, hiredate, grade, wage)
Student[personId] $\cap$ Employee[personId] = $\emptyset$

Students[personId] $\cup$ Employees[personId] returns all persons (view).

The University, Persons (Full, Overlapping Constraints)

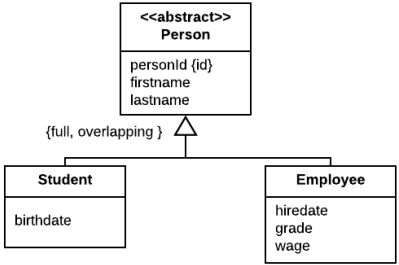
Focus on Person, Student and Employee with an “overlapping” constraint:



- Given a person p: p may be a student or an employee (or both)
- Define a relation Persons: attributes are those of the super-class
- Data related to a person who is student and employee are stored once only.

The University, Persons, Reference version

Focus on Person, Student and Employee with an “overlapping” constraint (we assume weak overlapping):

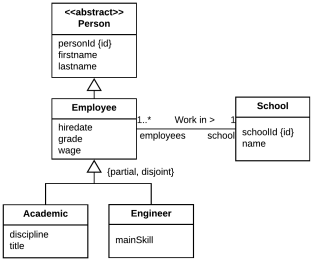


Persons (personId, first_name, last_name)
Students (personId, birthdate)
Employees (personId, hireDate, grade, wage)

Students[personId] $\subseteq$ Persons[personId]
Employees[personId] $\subseteq$ Persons[personId]

Students[personId] $\cup$ Employees[personId] = Persons[personId]

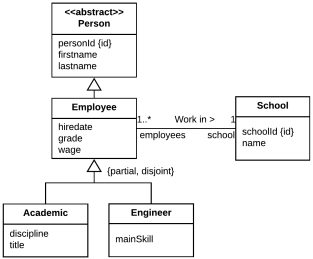
The University, Employees



As constraints are “partial” and “disjoint”:

- It may exist employees who are neither engineers nor academics. Thus, the relation Employees is necessary to store data about them.
- 2 options the relation Employees may represent:
  1. employees who are neither engineer nor academic (Duplication)
  2. all employees (Reference)

The University, Employees, Duplication version



Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
    / Contains only employees who are neither engineer nor academic. */*
Engineers (personId, hiredate, grade, wage, schoolId, mainSkill)
    / Contains tuples for engineers only. */*
Academics (personId, hiredate, grade, wage, schoolId, title, discipline)
    / Contains tuples for academics only. */*

The University, Employees, Duplication version. Referencial Integrity Constraints

Employees represents employees who are neither engineer nor academic

Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
    /* *Contains only employees who are neither engineer nor academic.* */
Engineers (personId, hiredate, grade, wage, schoolId, mainSkill)
    /* *Contains tuples for engineers only.* */
Academics (personId, hiredate, grade, wage, schoolId, title, discipline)
    /* *Contains tuples for academics only.* */
Employees[personId] $\subseteq$ Persons[personId]
Engineers[personId] $\subseteq$ Persons[personId]
Academics[personId] $\subseteq$ Persons[personId]
Engineers[personId] $\cap$ Employees[personId] = $\emptyset$
Academic[personId] $\cap$ Employees[personId] = $\emptyset$
Academics[personId] $\cap$ Engineers[personId] = $\emptyset$
schoolId not null in Employees, Engineers and Academics.

Why this solution (Duplication)?

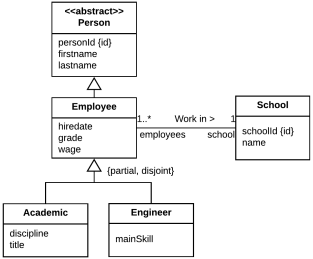
Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
/* *Contains only employees who are neither engineer nor academic.* */
Engineers (personId, hiredate, grade, wage, schoolId, mainSkill)
/* *Contains tuples for engineers only.* */
Academics (personId, hiredate, grade, wage, schoolId, title, discipline)
/* *Contains tuples for academics only.* */

In this solution (Duplication),

- Accessing all details of employees according to her/his role is privileged.
- To retrieve all employees an union between Employees, Engineers and Academis must be performed (via a view).

The University, Employees, Reference version

Employee represents all employees:



Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
/* *Contains one tuple for each employee.* */
Engineers (personId, mainSkill) /* *Engineers only.* */
Academics (personId, title, discipline) /* *Academics only.* */

The University, Employees, Reference version. Referential Integrity Constraints

Employee represents all employees

Employees (personId, hiredate, grade, wage, schoolId)
/ Contains one tuple for each employee. */*
Engineers (personId, mainSkill)
/ Engineers contains tuples for engineers only. */*
Academics (personId, title, discipline)
/ Academics contains tuples for academics only. */*
Employees[personId] $\subseteq$ Persons[personId]
Engineers[personId] $\subseteq$ Employees[personId]
Academics[personId] $\subseteq$ Employees[personId]
Engineers[personId] $\cap$ Academics[personId] = $\emptyset$
schoolId not null in Employees.

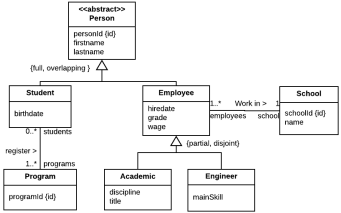
Why this solution (Reference)?

Employees (personId, hiredate, grade, wage, schoolId)
/ Contains one tuple for each employee. */*
Engineers (personId, mainSkill)
/ Engineers contains tuples for engineers only. */*
Academics (personId, title, discipline)
/ Academics contains tuples for academics only. */*

In this solution (Reference),

- Accessing all employees as a whole is privileged.
- Details of engineers (resp. academics) are retrieved with a natural join between relations Employees and Engineers (resp. Academics).

University: The global view of the transformation



+ Constraints defined before

Students (personId, first_name, last_name, birthdate)
Registrations (personId, programId);
Registrations[personId] = Students[personId] (1..*)
Registrations[programId] ⊆ Programs[programId] (0..*)
Programs (programId) -> Necessary because a Program can be unrealated with a student (0..*)
Schools (schoolId, name) Employees[schoolId] = Schools[schoolId]
Employees (personId, first_name, last_name, hiredate, grade, wage, schoolId)
Engineers (personId, mainSkill) *(Reference)* Engineers[personId] ⊆ Employees[personId]
Academics (personId, title, discipline) *(Reference)* Academics[personId] ⊆ Employees[personId]

8 Introduction

9 Mapping classes

10 Mapping Associations and Association classes

11 Mapping class Hierarchy

12 Concluding Remarks

- There is no theorem, nothing else than common sense!
- If something sounds wrong, go back to the conceptual design. Don't patch the relations!
- This is a "good" relational model
- Sometimes it needs a bit of clean up: check "useless" relations (=included in another one).

Chapter 5 – Relational Algebra

13 Introduction

14 Projection operator

15 Selection operator

16 Misc

17 Joining Relations

18 Set Operators: Union, Interaction and Substraction

19 Division

20 Conclusion

Relational algebra consists of

A set of operators that map relations to relations.

- Rules for combining those operations into expressions.
- Rules for evaluating such expressions.

An algebraic expression captures a query
Notations:

- R (X) and S (Y) are two relations where
- X and Y are two sets of attributes:
  $X = X1, ..., Xn$ $(n \geq 1)$ and $Y = Y1, ..., Yp$ $(p \geq 1)$

13 Introduction

14 Projection operator

15 Selection operator

16 Misc

17 Joining Relations

18 Set Operators: Union, Interaction and Substraction

19 Division

20 Conclusion

Projection (Examples)

Give the pseudo of all criminals

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

pseudo
El Profesor
Tokio
Nairobi
Denver
Rio
Berlin

Projection (Examples)

Give the pseudo of all criminals

Criminals[pseudo]

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

pseudo
El Profesor
Tokio
Nairobi
Denver
Rio
Berlin

Criminals[pseudo] is a **new relation** defined on attribute pseudo

For each criminal, find her(his) name and speciality

For each criminal, find her(his) name and speciality

Criminals[name,speciality]

/ defines a new relation whose attributes are name and speciality */*

Retrieve the birthyears

Retrieve the birthyears

Criminals[birthyear]

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

birthyear
1983
1989
1987
1995
1979

all values are pairwise different.

Projection - Definition

- *Notation:* R[A], A $\subseteq$ X
- *Relation schema:* A
- *Interpretation:* R[A] = { n[A] | n $\in$ R}, where n[A] is the projection of the tuple n on attributes A. Because R[A] is a set, all of the tuples are pairwise different.
- Also noted: $\pi_A(R)$

13 Introduction

14 Projection operator

15 Selection operator

16 Misc

17 Joining Relations

18 Set Operators: Union, Interaction and Substraction

19 Division

20 Conclusion

Selection (Examples)

Find the criminals that are Robbers

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	**Robber**	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	**Robber**	1979	2017052

Selection (Examples)

Find the criminals that are Robbers

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	**Robber**	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	**Robber**	1979	2017052

Criminals:speciality = 'Robber'

Find the criminals that are Robbers. Find also those who where born before 1985 and are involved in the crime 2017052

pseudo	**name**	**specialty**	**birthyear**	**crimeId**
El Profesor	Sergio	Band Leader	**1983**	**2017052**
Tokio	Silene	**Robber**	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	**Robber**	**1979**	**2017052**

Find the criminals that are Robbers. Find also those who where born before 1985 and are involved in the crime 2017052

pseudo	**name**	**specialty**	**birthyear**	**crimeId**
El Profesor	Sergio	Band Leader	**1983**	**2017052**
Tokio	Silene	**Robber**	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	**Robber**	**1979**	**2017052**

Criminals:(speciality = 'Robber' or (birthyear $<$ 1985 and crimeId = '2017052'))

Selection - Definition

- *Notation :* R:P where P is a predicate built as either:
  - A simple condition of the form:
    <att. name> <comp. op.> <att. name> or
    <att. name> <comp. op.> <cst. val.>
  - Or a complex boolean expression of the form:
    <cond.> <bool. op.> <cond.>
    where <cond.> is either a basic condition or a complex boolean expression.
  - Comparison operators (comp. op.): $=, \neq, <, >$,..
  - Boolean operators (bool. op.) (in **decreasing precedence**): not ($\neg$), and ($\wedge$), or ($\vee$)
  - P may contain brackets
- *Relation Schema:* X (same schema as R)
- *Interpretation:* R:P = $\{n \mid n \in R \wedge P(n)\}$
- Also noted: $\sigma_P(R)$

Boolean Expressions (some examples)

- A = B and B $>$ 1985
- A = 'Robber' or A = 'Burglar' and B $>$ 1985
- (A = 'Robber' or A = 'Burglar') and B $>$ 1985
- A = 'Robber' and B $>$ 1985 or A = 'Burglar' and B $>$ 1985
- not (A = 'Robber' or A = 'Burglar')
- A $\neq$ 'Robber' and A $\neq$ 'Burglar'

If you are not familiar with boolean expressions, get a book on Boole's logic and catch up!

13 Introduction

14 Projection operator

15 Selection operator

16 Misc

17 Joining Relations

18 Set Operators: Union, Interaction and Substraction

19 Division

20 Conclusion

Introducing New Names

The rename operation $\longleftarrow$ is the means to associate names to the result of a relational algebra expression. It is particularly useful for structuring a solution to a complex problem.

A $\longleftarrow$ Criminals:birthyear $\geq$ 1990

Res $\longleftarrow$ A[pseudo]

R (X,Y) $\longleftarrow$ A[pseudo,birthyear]

A

pseudo	name	specialty	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

Res

pseudo
Rio

R

X	Y
Rio	1995

Composition of Operations

Find the pseudo of the criminals whose birthyear is equal or greater than 1990

Criminals:birthyear $\geq$ 1990[pseudo]

Projection and selection have the same precedence, and therefore, they have left-to-right associativity.

Composition of Operations

Find the pseudo of the criminals whose birthyear is equal or greater than 1990

Criminals:birthyear $\geq$ 1990[pseudo]

Projection and selection have the same precedence, and therefore, they have left-to-right associativity.

Criminals[pseudo]:birthyear $\geq$ 1990

is incorrect because the projection applies first, and salary is not an attribute of Criminals[pseudo]

13 Introduction
14 Projection operator
15 Selection operator
16 Misc
17 Joining Relations
18 Set Operators: Union, Interaction and Substraction
19 Division
20 Conclusion

Cartesian Product

The cartesian is a means for combining two relations.

A	B
1	11
2	22

X

C	D
3	33
4	44

=

A	B	C	D
1	11	3	33
1	11	4	44
2	22	3	33
2	22	4	44

Criminals X Crimes is a relation whose schema is:

R (pseudo, name, speciality, birthyear, Criminals.crimeId, Crimes.crimeId, type, location)

/ As pseudo, name, speciality, birthyear, type and location appear in one relation only, the qualified notation can be omitted. */*

Cartesian Product

Criminals[pseudo, speciality, crimeId]

pseudo	**speciality**	**crimeId**
El Profesor	Band Leader	2017052
Tokio	Robber	2017052
Joker	Chemical Eng.	2019089

X

Crimes

crimeId	**type**	**location**
2017052	Heist	FNMT
2019089	Riots	Gotham
2003215	Heist	Antwerp

Cartesian Product

Criminals[pseudo, speciality, crimeId] X Crimes

pseudo	speciality	Criminals.crimeId	Crimes.crimeId	type	location
El Profesor	Band Leader	2017052	2017052	Heist	FNMT
Tokio	Robber	2017052	2017052	Heist	FNMT
Joker	Chemical Eng.	2019089	2017052	Heist	FNMT
El Profesor	Band Leader	2017052	2019089	Riots	Gotham
Tokio	Robber	2017052	2019089	Riots	Gotham
Joker	Chemical Eng.	2019089	2019089	Riots	Gotham
El Profesor	Band Leader	2017052	2003215	Heist	Antwerp
Tokio	Robber	2017052	2003215	Heist	Antwerp
Joker	Chemical Eng.	2019089	2003215	Heist	Antwerp

Cartesian Product

For each criminal (give pseudo and speciality) find the crimes (give the id and type) in which she(he) is involved

A $\longleftarrow$ Criminals[pseudo, speciality, crimeId] X Crimes[crimeId, type]
B $\longleftarrow$ A:Criminals.crimeId = Crimes.crimeId
R $\longleftarrow$ B[pseudo, speciality, Criminals.crimeId, type]

pseudo	speciality	Criminals.crimeId	Crimes.crimeId	type
El Profesor	Band Leader	2017052	2017052	Heist
Tokio	Robber	2017052	2017052	Heist
Joker	Chemical Eng.	2019089	2017052	Heist
El Profesor	Band Leader	2017052	2019089	Riots
Tokio	Robber	2017052	2019089	Riots
Joker	Chemical Eng.	2019089	2019089	Riots
El Profesor	Band Leader	2017052	2003215	Heist
Tokio	Robber	2017052	2003215	Heist
Joker	Chemical Eng.	2019089	2003215	Heist

Cartesian Product - Definition

- *Notation:* R x S
- *Relation schema:* $X \cup Y$ (assuming $X \cap Y = \emptyset$)
- *Interpretation:* R x S = {<r, s> | r $\in$ R and s $\in$ S}
- *Special case:* X = {A, B} and Y ={B, C}
  When R1 and R2 have attributes in common, fully qualified attribute names have to be used:
  Schema(R1 (A, B) x R2 (B, C)) is {A, R1.B, R2.B, C}

Relational/Theta Join

For each criminal (give pseudo and speciality) find the crimes (give the id and type) in which she(he) is involved(again!)

A $\longleftarrow$ Criminals[pseudo, speciality, crimeId](Criminals.crimeId=Crimes.crimeId)*Crimes[crimeId, type
R $\longleftarrow$ A[pseudo, speciality, Criminals.crimeId, type]

pseudo	**speciality**	**Criminals.crimeId**	**Crimes.crimeId**	**type**
El Profesor	Band Leader	2017052	2017052	Heist
Tokio	Robber	2017052	2017052	Heist
Joker	Chemical Eng.	2019089	2019089	Riots

Relational/Theta Join - Definition

- *Notation:* R(P)*S
  P is a boolean expression whose **operands are attribute names only**. Simple boolean expressions are of the form Xi θ Yi where Xi is an attribute of R and Yi an attribute of S, and $\theta \in \{=, \neq, <, >, ...\}$
- *Relation schema:* $X \cup Y$ (assuming $X \cap Y = \emptyset$)
- *Interpretation:*
  R(P)*S = {<r, s> | r $\in$ R and s $\in$ S and P(<r,s>)}
- *Property:* R(P)*S = (R X S):P
- Also noted: $R \bowtie_P S$

Natural Join

When the relational join involves equality comparison only on *attributes whose names are identical* it is called *natural join*. In the schema of the result, **superfluous attributes have been removed**.

For each criminal (give pseudo and speciality) find the crimes (give the id and type) in which she(he) is involved(again!)

R $\longleftarrow$ Criminals[pseudo, speciality, crimeId]*Crimes[crimeId, type]

pseudo	**speciality**	**crimeId**	**type**
El Profesor	Band Leader	2017052	Heist
Tokio	Robber	2017052	Heist
Joker	Chemical Eng.	2019089	Riots

Natural Join - Definition

- Assumption: R (X_1, ..., X_n) and S (X_p, ..., X_m) where:
  A = {X_1, ..., X_n} − {X_p, ..., X_m} (attributes in R only)
  B = {X_1, ..., X_n} ∩ {X_p, ..., X_m} (attributes in both R and S)
  C = {X_p, ..., X_m} − {X_1, ..., X_n} (attributes in S only)
- *Notation:* R * S
- *Relation schema:* {X_1, ..., X_n} ∪ {X_p, ..., X_m}
- *Interpretation:* R * S = {<r[A], r[B], s[C]> | r ∈ R ∧ s ∈ S ∧ r[B] = s[B] }
- *Property:* R * S = (R(R.B1=S.B1, ..., R.Bi=S.Bi)*S)[A, B, C]
  where B = {B1, .., Bi}
  B = ∅ $\Longrightarrow$ R * S = ∅

Exercice

Give the pseudo of the Band Leaders that participated in at least one Heist

Criminals (pseudo, name, speciality, birthyear, crimeId)
Crimes(crimeId, type, location)

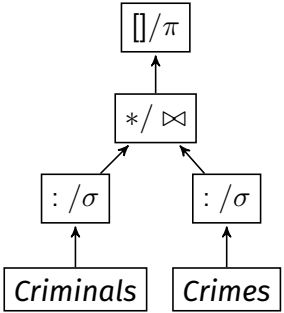
Exercice

Give the pseudo of the Band Leaders that participated in at least one Heist

Criminals (pseudo, name, speciality, birthyear, crimeId)
Crimes(crimeId, type, location)

Plan

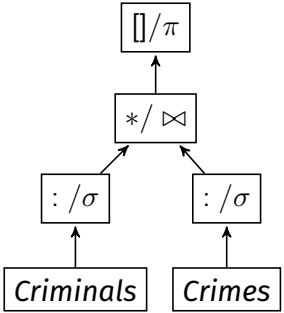
(Criminals:speciality='Band Leader'*Crimes:type='Heist')[pseudo]



Exercice

Give the pseudo of the Band Leaders that participated in at least one Heist

Criminals (pseudo, name, speciality, birthyear, crimeId)
Crimes(crimeId, type, location)



Plan

(Criminals:speciality='Band Leader'*Crimes:type='Heist')[pseudo]

```
SELECT DISTINCT pseudo
FROM Criminals JOIN Crimes USING (crimeId)
WHERE speciality='Band Leader' AND type='Heist';
```

Multiple Citation of Relations

For the criminal 'El Profesor' find his accomplices (give the pseudo(s))

A ⟵ Criminals:pseudo = 'El Profesor'[pseudo, crimeId]
B(pc1, cc1, pc2, cc2) ⟵
    A (A.crimeId=Criminals.crimeId and A.pseudo≠Criminals.pseudo)*Criminals[pseudo, crimeId]
R ⟵ B[pc2]

A

pseudo	crimeId
El Profesor	2017052

Criminals[pseudo, crimeId]

pseudo	crimeId
El Profesor	2017052
Tokio	2017052
Joker	2019089

B

pc1	cc1	pc2	cc2
El Profesor	2017052	Tokio	2017052

Because Criminals is used twice, the intermediate relation is mandatory: there is no straight expression!

13 Introduction

14 Projection operator

15 Selection operator

16 Misc

17 Joining Relations

18 Set Operators: Union, Interaction and Substraction

19 Division

20 Conclusion

Set Operators

- *Interpretation:*
  $R \cup S = \{ t \mid t \in R \text{ or } t \in S\}$
  $R - S = \{t \mid t \in R \text{ and } t \notin S\}$
  $R \cap S = \{ t \mid t \in R \text{ and } t \in S\}$
- *Property:* $R \cap S = R - (R - S) = S - (S - R)$

Set Operators - Definition

- *Notation:* $R(X) \cup S(Y)$, $R(X) \cap S(Y)$, $R(X) - S(Y)$
  $X = X_1, .., X_n$ and $Y = Y_1, .., Y_p$ and $n=p$ and
  $\forall i \in [1, .., n]$, domain (X_i) and domain (Y_i) are comparable.
- *Relation schema:* when the attribute names of R and S are pairwise identical the result is a relation with the same schema, otherwise the attribute names of the result must be explicity defined using the rename operation.
  For example: $T(Z_1, Z_2, .., Z_n) \longleftarrow R \cup S$, where:
  $\forall i \in [1, .., n]$, domain (Z_i) = domain (X_i) = domain (Y_i)

Example Minus

Retrieve the crimes with no criminals involved in

A $\longleftarrow$ Crimes[crimeId] − Criminals[crimeId]
R $\longleftarrow$ A * Crimes

Crimes

crimeId	type	location
2017052	Heist	FNMT
2019089	Riots	Gotham
2003215	Heist	Antwerp

Criminals

pseudo	name	speciality	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Joker	Arthur	Chemical Eng.	1985	2019089

R

crimeId	type	location
2003215	Heist	Antwerp

Question: Why is the following expression incorrect for the same query?

(Crimes(Crimes.crimeId $\neq$ Criminals.crimeId)*Criminals)[Crimes.crimeId, type, location]

Question: Why is the following expression incorrect for the same query?

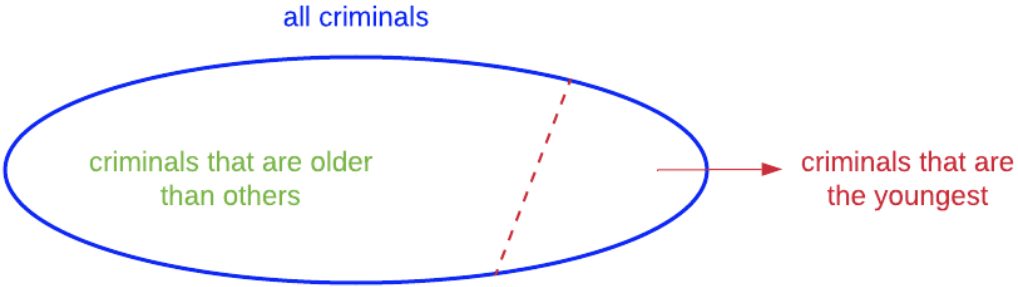
(Crimes(Crimes.crimeId $\neq$ Criminals.crimeId)*Criminals)[Crimes.crimeId, type, location]

Pairwise comparison (see below)

pseudo	**...**	**Criminals.crimeId**	**Crimes.crimeId**	**type**	**location**
El Profesor	...	2017052	2017052	Heist	FNMT
Tokio	...	2017052	2017052	Heist	FNMT
Joker	...	2019089	2017052	Heist	FNMT
El Profesor	...	2017052	2019089	Riots	Gotham
Tokio	...	2017052	2019089	Riots	Gotham
Joker	...	2019089	2019089	Riots	Gotham
El Profesor	...	2017052	2003215	Heist	Antwerp
Tokio	...	2017052	2003215	Heist	Antwerp
Joker	...	2019089	2003215	Heist	Antwerp

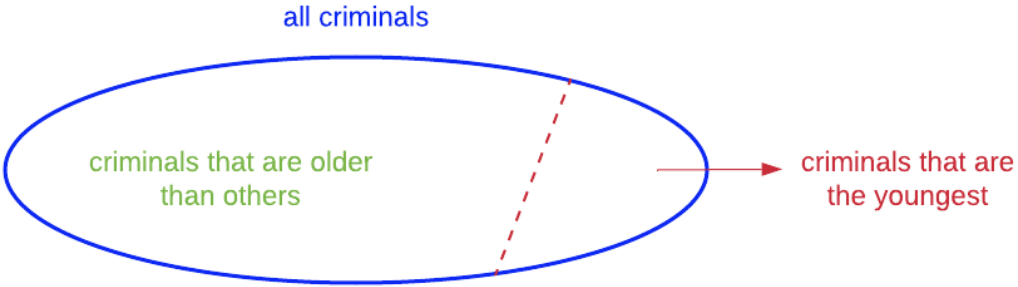
Example 2

Retrieve the pseudo of the youngest criminal



Example 2

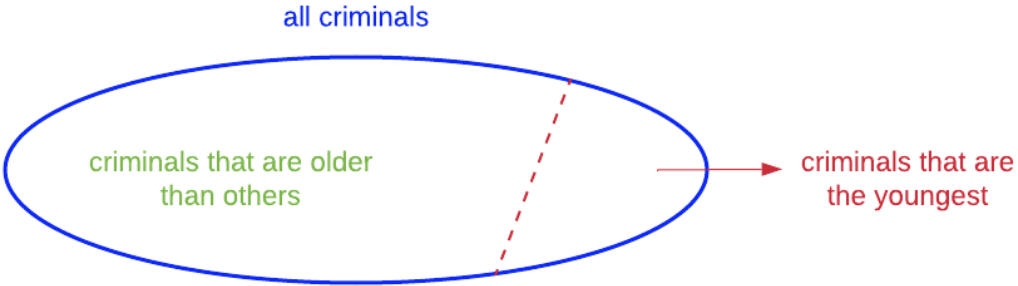
Retrieve the pseudo of the youngest criminal



C1(p, y) ⟵ Criminals[pseudo, birthyear]
C2(p, y) ⟵ Criminals[pseudo, birthyear]

Example 2

Retrieve the pseudo of the youngest criminal



C1(p, y) $\longleftarrow$ Criminals[pseudo, birthyear]
C2(p, y) $\longleftarrow$ Criminals[pseudo, birthyear]
OlderThanOther $\longleftarrow$ (C1(C1.y < C2.y)*C2)[C1.p]
/* <x> $\in$ *OlderThanOther* $\Longleftrightarrow$ *x is a criminal that is older than somebody else.* */
TheYoungest $\longleftarrow$ C1[p] − OlderThanOther

Example 2 (with summarized data)

C1(p, y) $\longleftarrow$ Criminals[pseudo, birthyear]
C2(p, y) $\longleftarrow$ Criminals[pseudo, birthyear]
OlderThanOther $\longleftarrow$ (C1(C1.y $<$ C2.y)*C2)[C1.p]
/* <x> $\in$ *OlderThanOther* $\Longleftrightarrow$ *x is a criminal that is older than somebody else.* */
TheYoungest $\longleftarrow$ C1[p] $-$ OlderThanOther

Criminals

pseudo	name	speciality	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Joker	Arthur	Chemical Eng.	1985	2019089

C1

p	y
El Profesor	1983
Tokio	1989
Joker	1985

C2

p	y
El Profesor	1983
Tokio	1989
Joker	1985

OlderThanOther

p
El Profesor
Joker

TheYoungest

p
Tokio

13 Introduction
14 Projection operator
15 Selection operator
16 Misc
17 Joining Relations
18 Set Operators: Union, Interaction and Substraction
19 Division
20 Conclusion

Motivation

Enrolls (student, sport) (the key is <student,sport>). *Find students who are enrolled in all sports*

All ⟵ Enrolls[student] X Enrolls[sport] /* *all possible student - sport.* */
NotAll ⟵ (All − Enrolls)[student]
/* *students who aren't enrolled in at least one sport* */
Res ⟵ Enrolls[student] − NotAll
/* *students who are enrolled in all sports* */
In short: Enrolls / Enrolls[sport]

Enrolls

student	sport
John	Box
Tom	Judo
Mary	Judo
Tom	Box

All

student	sport
John	Box
John	Judo
Mary	Box
Mary	Judo
Tom	Box
Tom	Judo

NotAll

student	sport
John	Judo
Mary	Box

Res

student
Tom

Another example

Retrieve the pseudo of the youngest criminal

Another example

Retrieve the pseudo of the youngest criminal

C1 $\longleftarrow$ Criminals; C2 $\longleftarrow$ Criminals
YoungerOrEqual(p1,p2) $\longleftarrow$ (C1(C1.birthyear $\geq$ C2.birthyear)*C2)[C1.pseudo,C2.pseudo]
/* <x,y> $\in$ *YoungerOrEqual* $\Longleftrightarrow$ *x's birthyear is higher or equal than y's birthyear* */
TheYoungest $\longleftarrow$ YoungerOrEqual / YoungerOrEqual[p2]

Criminals

pseudo	name	speciality	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Joker	Arthur	Chemical Eng.	1985	2019089

YoungerOrEqual

p1	p2
El Profesor	El Profesor
Tokio	El Profesor
Tokio	Tokio
Tokio	Joker
Joker	El Profesor
Joker	Joker

TheYoungest

p1
Tokio

Tokio is the only value on p1 that matches with all possible values of p2.

Chapter 5 – Relational Algebra

13 Introduction

14 Projection operator

15 Selection operator

16 Misc

17 Joining Relations

18 Set Operators: Union, Interaction and Substraction

19 Division

20 Conclusion

Precedence of Operators

The normal way to group operators is, from the highest precedence to the lowest:

- *Unary operators :* Projection, Selection
- *"Multiplicative" operators :* Cartesian Product, Natural Join, Theta Join, Division
- *"Additive" operators :* Difference, Union, Intersection

But there is no universal agreement, so it is a good idea to group all binary operators with parentheses enclosing their arguments.
All operators have left-to-right associativity. Binary operators, except set difference, are commutative.

Concluding Remarks

Isn't this a bit theoretical?

- High level notation lets us focus on reasoning about relations.
- Relational algebra defined the basics of SQL (we'll study SQL later).
- SQL queries are mapped to relational expressions during the query optimization process.

21 Introduction

Preliminaries

SQL is an ANSI/ISO standard language for querying and manipulating relational data. Designed to be a "human readable" language comprising:

- Data definition facilities
- Database modification operations
- Relational algebra operations
- Aggregation operations

Preliminaries

SQL is an ANSI/ISO standard language for querying and manipulating relational data. Designed to be a "human readable" language comprising:

- Data definition facilities
- Database modification operations
- Relational algebra operations
- Aggregation operations

It is easy to express simple queries
VERY tricky to express complex queries!

SQL: history

- SQL was developed at IBM during the 1970's, and standardised during the 80's.
- SQL-92 the previous standard
- SQL:1999 the current standard, also called SQL-99
- SQL:2003 was supposed to extend the language with objects
- SQL:2006 defines ways that SQL can be used with XML
- SQL:2008 add of new statements (MERGE, TRUNCATE)
- SQL: 2011 adds temporal data (PERIOD FOR)
- SQL: 2016 Adds row pattern matching, polymorphic table functions, JSON

Most systems extend the standard language in various dialects.

22 Introduction

23 Integrity constraints

24 SQL Types

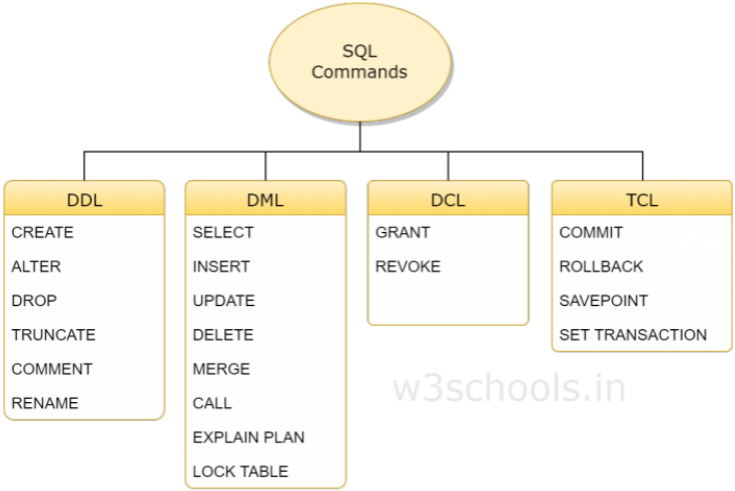
25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

SQL Commands - These slides concerns...



- DDL – Data Definition Language (CREATE, ALTER, DROP)
- DML – Data Manipulation Language (INSERT, DELETE, UPDATE)
- DCL – Data Control Language
- TCL - Transaction Control Language

Aims of SQL DDL

Defining data relies on SQL Data Definition Language part, that is the means to:

- Create and alter database schema: tables, views, users, ...,
- Insert and modify data
- DDL statements are submitted as a query (via SQL interpreter or from an application program).

If any of the results violates an integrity constraint
the DBMS rolls back the statement and raises an exception.
The database is then kept consistent.

22 Introduction

23 Integrity constraints

24 SQL Types

25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

Data Integrity

Integrity of data relies on:

- *Integrity constraints* which:
  - prevent invalid data entry into the database.
  - enforce the business rules associated with the information in a database.
- *Concurrent access control* (not studied in this course)

Integrity Constraints Typology

One table, one database state

- Value restriction:
  The birth year must be greater than 1900
- Mandatory / Uniqueness:
  The pseudo cannot be missing and they are pairwise different
- Identification:
  Crimes are uniquely identified by their id.

Integrity Constraints Typology

One table, one database state

- Value restriction:
  The birth year must be greater than 1900
- Mandatory / Uniqueness:
  The pseudo cannot be missing and they are pairwise different
- Identification:
  Crimes are uniquely identified by their id.

Many tables, one database state

- Referential Integrity: a value (a projection on an attributes or set of attributes) in one relation must match at least a value in a related relation.
  Each criminal is involved in an existing crime.
- Value restriction:
  Each criminal involved in a crime should be born before the crime is committed.

Many tables, many database states

- Value restriction:
  A criminal's years of sentence cannot increase.
- Value restriction:
  A criminal may have a conditional freedom after fulfilling one third of the sentence.

Advantages of Integrity Constraints

when specified in SQL and enforced by the DBMS...

- **Declarative Ease (SQL query)**
  No additional programming is required. The DBMS keeps the control.
- **Centralised Rules**
  Integrity constraints are defined for tables (not an application). In case of business rules modification, the associated integrity constraints need only to be changed (applications automatically adhere to the modified constraints).
- **Superior Performance**
  The semantics of integrity constraint declarations are clearly defined, and performance optimisations are implemented for each specific declarative rule.
- **Flexibility**
  Integrity constraints can be temporarily disabled to fit specific needs (bulk loads).
  Beware: when a constraint is disabled it is so for all applications!

Easier to manage than application-based integrity constraints

Concluding remarks

- Most of the DBMS support only a few of these integrity constraints (one relation, one database state).
- Integrity constraints which are not supported by the DBMS must be enforced at the application level (or by triggers when possible).
- Enforcing all others must be left to the DBMS... but according to the way the user interface has been designed, some integrity are **also** handled at the interface layer:
  - Value type and restriction (type of program variable)
  - Enumeration and referential integrity (limited choice given by a menu)
  - Etc.

Good practice: it is worth using the most the DBMS features.

22 Introduction

23 Integrity constraints

24 SQL Types

25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

SQL Main Data Types - (RDBMS dependant)

Note that SQLite has dynamic typing :
`https://www.sqlite.org/datatype3.html`

- **Strings in Oracle**:
  CHAR [(n)]: uses n characters (fixed length), left-justified, blank-padded
  VARCHAR [(n)]: uses 0..n characters (variable length), no padding
- **Strings in SQLite**:
  TEXT
- **Numbers in Oracle**
  number (p, s): a fixed-point number with precision p and scale s
  number (p): a fixed-point number with precision p
  number: a floating-point number with decimal precision 38
- **Numbers in SQLite**
  INTEGER / REAL (FLOAT)

SQL Main Data Types - (RDBMS dependant)

Note that SQLite has a specific way of dealing with dates (saved as INT or TEXT):
`https://www.sqlite.org/lang_datefunc.html`

- **Date in Oracle**
  An instant (in Oracle: unit second;)
  Default format : DD-MON-YY. Ex: '22-NOV-2022'

  Date in SQlite
  Default format : YYYY-MM-DD. Ex. '2022-11-22'
- **Others**
  Long, Raw, Blob, etc.

Operators and functions (RDBMS dependant)

- **Comparison** operators (on all types):
  $<, >, <>, !=, =$
- **Boolean** operators:
  NOT, AND, OR (given in decreased order of precedence)
- **Arithmetic** operators:
  + Addition, - Subtraction, * Multiplication, / Division
- **Other** numeric operators:
  abs, ceil, floor, trunc, round, exp, pow, mod, sqrt, etc.
  sin, cos, tan, atan, etc.
- **String** functions: concat, lower, upper, substring, length, to_number, etc.

- **Date functions in Oracle**:
  to_date (string,string) $\longrightarrow$ a date
  / to_date ('05 Dec 2001', 'DD Mon YYYY') */*
  to_char (date, string) $\longrightarrow$ a string
  / to_char (d,f) returns a string representing d according to the format f. */*
  / to_char (to_date ('05 Dec 2001', 'DD Mon YYYY'), 'DD/MM/YY') = '05/12/01' */*
- Some other date functions:
  add_month (date, integer) $\longrightarrow$ a date
  / add_months (d1, n) returns the date d plus n months. */*
- Arithmetic operators extended to dates:
  d1, d2: date, n: integer ≥ 0
  d1 - d2 is the number of days between d1 and d2
  d1 + n is the date n days in front of d1
  d1 - n is the date n days behind d1
- Other functions: SYSDATE $\longrightarrow$ a date, etc.

22 Introduction

23 Integrity constraints

24 SQL Types

25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

Creating relations

To create (empty) relations:

```
CREATE TABLE T (
    List_of_attribute_definitions
     [,List_of_constraint_definitions ]
) ;
```

List of attribute definitions:

- *A1 number (3),*
- *A2 VARCHAR (10),*
- *A3 number (3) default 1,*
- ...

Constraint specifications in SQL

CONSTRAINT c1 CHECK (P)
the CONSTRAINT c1 validates iff P is true or null
l: *possibly many check statements per relation.*
A4 date CONSTRAINT c2 NOT NULL
NOT NULL constraint must be specified with attribute definition.
CONSTRAINT c3 UNIQUE (A3)
CONSTRAINT c4 PRIMARY KEY (A1, A2)
only one PRIMARY KEY per relation.
CONSTRAINT c5 FOREIGN KEY (A1,..,An) REFERENCES Target(B1,..,Bn)
B1,...,Bn must be the PRIMARY KEY in Target.
B1,..,Bn must be compatible with A1,..,An

For a comprehensive description of the syntax, do not hesitate to consult the SQL reference guide!

A first example

We consider the toy database:

Bars(name, addr, openDate)
Drinker(name, addr, phone)
Frequents(drinker,bar)

Frequents[bar] $\subseteq$ Bars[name]
Frequents[drinker] $\subseteq$ Drinkers[name]

```
CREATE TABLE Bars (
  name VARCHAR(30) PRIMARY KEY,
  addr VARCHAR(30),
  openDate date
);

CREATE TABLE Drinkers (
  name VARCHAR(30),
  addr VARCHAR(30),
  phone VARCHAR(30),
  CONSTRAINT Drinkers_c0 PRIMARY KEY (name)
);

CREATE TABLE Frequents (
  drinker VARCHAR(30),
  bar VARCHAR(30),
  CONSTRAINT Frequents_c0 PRIMARY KEY (drinker,bar),
  CONSTRAINT Frequents_c1 FOREIGN KEY (bar)
                 REFERENCES bars(name),
  CONSTRAINT Frequents_c2 FOREIGN KEY (drinker)
                 REFERENCES drinkers(name)
);
```

Inserts

```
SQL> INSERT INTO Bars VALUES ('Le bon coin', 'Grenoble',
to_date ('29/02/2011','DD/MM/YYYY'));
```

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble');
```

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble', 'rrrrr');
```

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Voiron', 'aaaaa');
```

Inserts

```
SQL> INSERT INTO Bars VALUES ('Le bon coin', 'Grenoble',
to_date ('29/02/2011','DD/MM/YYYY'));
```

ERROR at line 1: ORA-01839: date not valid for month specified

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble');
```

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble', 'rrrrr');
```

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Voiron', 'aaaaa');
```

Inserts

```
SQL> INSERT INTO Bars VALUES ('Le bon coin', 'Grenoble',
to_date ('29/02/2011','DD/MM/YYYY'));
```

ERROR at line 1: ORA-01839: date not valid for month specified

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble');
```

ERROR at line 1: ORA-00947: not enough values

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble', 'rrrrr');
```

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Voiron', 'aaaaa');
```

Inserts

```
SQL> INSERT INTO Bars VALUES ('Le bon coin', 'Grenoble',
to_date ('29/02/2011','DD/MM/YYYY'));
```

ERROR at line 1: ORA-01839: date not valid for month specified

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble');
```

ERROR at line 1: ORA-00947: not enough values

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble', 'rrrrr');
```

1 row created.

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Voiron', 'aaaaa');
```

Inserts

```
SQL> INSERT INTO Bars VALUES ('Le bon coin', 'Grenoble',
to_date ('29/02/2011','DD/MM/YYYY'));
```

ERROR at line 1: ORA-01839: date not valid for month specified

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble');
```

ERROR at line 1: ORA-00947: not enough values

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Grenoble', 'rrrrr');
```

1 row created.

```
SQL> INSERT INTO drinkers VALUES ('Pierre', 'Voiron', 'aaaaa');
```

ERROR at line 1: ORA-00001: unique constraint (BEERS.D_Co) violated

What happens when foreign keys apply?

```
SQL> INSERT INTO Frequents VALUES ('Pierre', 'Bar les amis');
```

```
SQL> INSERT INTO Frequents VALUES ('Paul', 'Coogee Bay Hotel');
```

What happens when foreign keys apply?

```
SQL> INSERT INTO Frequents VALUES ('Pierre', 'Bar les amis');
```

ERROR at line 1: ORA-02291: integrity constraint (BEERS.F_C1) violated - parent key not found

```
SQL> INSERT INTO Frequents VALUES ('Paul', 'Coogee Bay Hotel');
```

What happens when foreign keys apply?

```
SQL> INSERT INTO Frequents VALUES ('Pierre', 'Bar les amis');
```

ERROR at line 1: ORA-02291: integrity constraint (BEERS.F_C1) violated - parent key not found

```
SQL> INSERT INTO Frequents VALUES ('Paul', 'Coogee Bay Hotel');
```

ERROR at line 1: ORA-02291: integrity constraint (BEERS.F_C2) violated - parent key not found

Rules associated with referential integrity constraints

Provide users a mean to specify how the DBMS should behave when modifying data involved in foreign keys.

Exemple: Frequents[Drinker] $\subseteq$ Drinkers[Name]

Drinkers

Name	Addr
Adam	Randwick
Gernot	Newtown
John	Clovelly
Justin	Mosman

Frequents

Drinker	Bar
Adam	Coogee Bay Hotel
Gernot	Lord Nelson
John	Coogee Bay Hotel
John	Lord Nelson
John	Australia Hotel
Justin	Regent Hotel
Justin	Marble Bar

- Delete <John, Clovelly> FROM Drinkers?
- Modify <Adam, Randwick> in <Adrian, Randwick>?
- Add <Marie, Regent Hotel> in Frequents?

```
SQL> DELETE FROM Drinkers WHERE name='John';
```

```
SQL> UPDATE Drinkers SET name='Adrian' WHERE name='Adam';
```

```
SQL> DELETE FROM Drinkers WHERE name='John';
```

ERROR at line 1: ORA-02292: integrity constraint (BEERS.F_C2) violated - child record found

```
SQL> UPDATE Drinkers SET name='Adrian' WHERE name='Adam';
```

```
SQL> DELETE FROM Drinkers WHERE name='John';
```

ERROR at line 1: ORA-02292: integrity constraint (BEERS.F_C2) violated - child record found

```
SQL> UPDATE Drinkers SET name='Adrian' WHERE name='Adam';
```

ERROR at line 1: ORA-02292: integrity constraint (BEERS.F_C2) violated - child record found

Rules are:

- **Restrict**: disallows the update or deletion of referenced data (default).
- **Set to null (resp. default)**: when referenced data is updated or deleted, all associated dependent data is set to NULL (resp. a default value).
- **Cascade**: when referenced data is updated, all associated dependent data is correspondingly updated; when a referenced row is deleted, all associated dependent rows are deleted.

Example: relation Frequents

```
CREATE TABLE Frequents (
    ....
    CONSTRAINT fk_drinker FOREIGN KEY (drinker) REFERENCES Drinkers(name)
        ON DELETE CASCADE ON UPDATE CASCADE
    ... );
```

- No on .. cascade $\Longrightarrow$ restrict
- ON UPDATE CASCADE $\Longrightarrow$ "Adam" is changed with "Adrian" in Frequents
- ON DELETE CASCADE $\Longrightarrow$ all tuples linked to "John" are deleted from Frequents

Example: the library

Consider the library application:

Books (title, editionDate)
Items (itemId, purchaseDate, title)
    title NOT NULL
    Items[title] = Books[title]
Loans (itemId, loanDate, returnDate, memberId)
    / *itemId,returnDate is another key* */*
        loanDate <= returnDate
        memberId NOT NULL
        Loans[itemId] $\subseteq$ Items[itemId]
domain (loanDate) = domain (returnDate) = Date(Hour)
domain (purchaseDate) = domain (editionDate) = Date(Day)
Other integrity Constraints:
    Each member cannot have more than 4 loans at a time
    Each item is borrowed none or once each time

The library: SQL code

```
CREATE TABLE Books (
   title VARCHAR(30) PRIMARY KEY,
   editionDate DATE
);
CREATE TABLE Items (
   itemId VARCHAR(10) PRIMARY KEY,
   purchaseDate DATE,
   title VARCHAR(30) CONSTRAINT Items_c0 NOT NULL,
   CONSTRAINT Items_c1 FOREIGN KEY (title) REFERENCES Books (title)
            ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE Loans (
   itemId VARCHAR(10),
   loanDate DATE,
   returnDate DATE,
   memberId VARCHAR(10) CONSTRAINT Loans_c0 NOT NULL,
   CONSTRAINT Loans_c1 PRIMARY KEY (itemId, loanDate),
   CONSTRAINT Loans_c2 UNIQUE (itemId, returnDate),
   CONSTRAINT Loans_c3 FOREIGN KEY (itemid) REFERENCES Items (itemId)
            ON DELETE CASCADE ON UPDATE CASCADE,
   CONSTRAINT Loan_c4 CHECK (loanDate<=returnDate)
);
```

The library: which constraints still have to be enforced by programs?

- Each member cannot have more than 4 loans at a time
- Each item is borrowed none or once each time
- How about Items[title] = Books[title]?

The Campus

Buildings (buildingId, caretaker)
Floors (floorId, buildingId, access)
Rooms (roomId, floorId, buildingId, capacity)
Floors[buildingId] = Buildings[buildingId]
Rooms[floorId, buildingId] = Floors[floorId, buildingId]

```
CREATE TABLE Buildings (
   buildingId number(4),
   caretaker VARCHAR(20),
   CONSTRAINT Buildings_c1 PRIMARY KEY (buildingId)
);
CREATE TABLE Floors (
   floorId number(4),
   buildingId number(4),
   access VARCHAR(20),
   CONSTRAINT Floors_c1 PRIMARY KEY (floorId,buildingId),
   CONSTRAINT Floors_c2 FOREIGN KEY (buildingId)
      REFERENCES Buildings (buildingId)
      ON DELETE CASCADE ON UPDATE CASCADE
);
CREATE TABLE Rooms (roomId number(4),
   floorId number(4),
   buildingId number(4),
   capacity number(3),
   CONSTRAINT Rooms_c1 PRIMARY KEY (floorId,buildingId,roomId),
   CONSTRAINT Rooms_c2 FOREIGN KEY (floorId,buildingId)
      REFERENCES Floors (floorId,buildingId)
      ON DELETE CASCADE ON UPDATE CASCADE
   CONSTRAINT Rooms_c3 CHECK (capacity > 0)
);
```

How about integrity constraints?

Floors[buildingId] $\subseteq$ Buildings[buildingId]
is handled, but not
Buildings[buildingId] $\subseteq$ Floors[buildingId]

Similarly,
Rooms[floorId, buildingId] $\subseteq$ Floors[floorId, buildingId]
is handled while
Floors[floorId, buildingId] $\subseteq$ Rooms[floorId, buildingId]
is not

Hence, constraints below have to be enforced at application layer:

Buildings[buildingId] $\subseteq$ Floors[buildingId]
Floors[floorId, buildingId] $\subseteq$ Rooms[floorId, buildingId]

Constraint specifications: good practice!

Integrity constraints must be named

- Schema maintenance purpose: to allow constraint disabling/enabling
- Programming purpose: to ease exception interpretation and enhance messages returned.
- Beware of the naming rules which apply in the DBMS you wish to use

Schema Definition: Summary

- Beware of table definition order!
- Some integrity constraints remain in charge of applications:
- Options “ON DELETE CASCADE”, “ON UPDATE CASCADE” could be used (be careful, some DBMS does not implement all of them)

22 Introduction

23 Integrity constraints

24 SQL Types

25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

Dropping tables

DROP TABLE T; drops the table T and its tuples

In SQL:

```
DROP TABLE Loans ;
```

If T contains any tuples, linked to any tables (because of referential integrity constraints) the operation will be rejected unless the option "CASCADE" is present.

Altering tables

It is possible to perform any modification which does not lead to any loss of information.

- Add a column (all existing tuples have null value for this new column)
- Add an integrity constraint
- Redefine a column (datatype, size, default value)
- Disable, or drop an integrity constraint
- Enable an integrity constraint (possible only if the relation's tuples evaluate it to true).

Some examples:

In SQL:

```
ALTER TABLE Students add (birthday DATE, address VARCHAR(20)) ;
ALTER TABLE Students modify (address VARCHAR (30)) ;
ALTER TABLE Students disable CONSTRAINT Students_c1 ;
```

Inserting Data

- *INSERT INTO table_name VALUES ...*

Two options: *One tuple at a time*

In SQL:

```
INSERT INTO Drinkers VALUES ('Tom', 'Limoges', null) ;
INSERT INTO Bars VALUES ('The New Irish Pub', 'Limoges', 000001, SYSDATE) ;
```

From an existing relation

In SQL:

```
INSERT INTO Drinkers
SELECT A, B, C FROM R WHERE .....
```

These modifications are successfully executed only if they satisfy all constraints specified in the relation

Updating data

- *UPDATE table_name*
  SET attribute_name1 = new_value1, ... attribute_nameN = new_valueN
  WHERE condition ; — the WHERE clause is optional

Example:

In SQL:

```
UPDATE Rooms
    SET capacity = capacity * 0.8
    WHERE capacity >= 12 ;
```

Deleting data

- *DELETE FROM table_name [WHERE condition];*

Example :

In SQL:

```
DELETE FROM Buildings ;
```

delete all tuples in all relations Buildings, Floors and Rooms (because of the option *ON DELETE CASCADE*)

22 Introduction

23 Integrity constraints

24 SQL Types

25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

Creating Views

A view is a table whose data are *dynamically* retrieved by a query:

In SQL:

```
CREATE VIEW Persons (personId, firstname, lastname, age) AS
    SELECT personId, firstname, lastname,
        TRUNC(MONTHS_BETWEEN(SYSDATE, birthday)/12) AS age
    FROM Employees
    UNION
    SELECT studentId, firstname, lastname,
        TRUNC(MONTHS_BETWEEN(SYSDATE, birthday)/12) AS age
    FROM Students;
```

A view may be thought as

- a customised presentation of the data stored in one or more tables
- a way to model derived attributes and avoid having to compute them in applications

Advantages of views

Views are often used for:

- Hiding data complexity
- Presenting data for different perspectives
- Providing an additional granularity level of confidentiality

A view (like tables) can be

- queried,
- updated with some restrictions
  (updates are propagated into the tables the view relies on, see later)

Example

In SQL:

```
/* <m, n> ∈ HowManyLoans ⟺ the member identified by m currently holds n loans. */
CREATE VIEW HowManyLoans (memberId, loanNb) AS
    SELECT memberId, count (itemId)
    FROM Loans
    WHERE returnDate IS NULL
    GROUP BY memberId;
```

Retrieve members who currently have 4 loans: return also the ids and titles of borrowed items.

In SQL:

```
SELECT memberId, itemId, title
FROM HowManyLoans JOIN Loans USING (memberId) JOIN Items USING (itemId)
WHERE loanNb = 4;
```

Dropping views

DROP VIEW V; delete the view V (doesn't modify any of the related relations.)

In SQL:

```
DROP VIEW HowManyLoans;
```

22 Introduction

23 Integrity constraints

24 SQL Types

25 Creating relations

26 Altering and dropping relations

27 Creating and dropping views

28 Conclusion

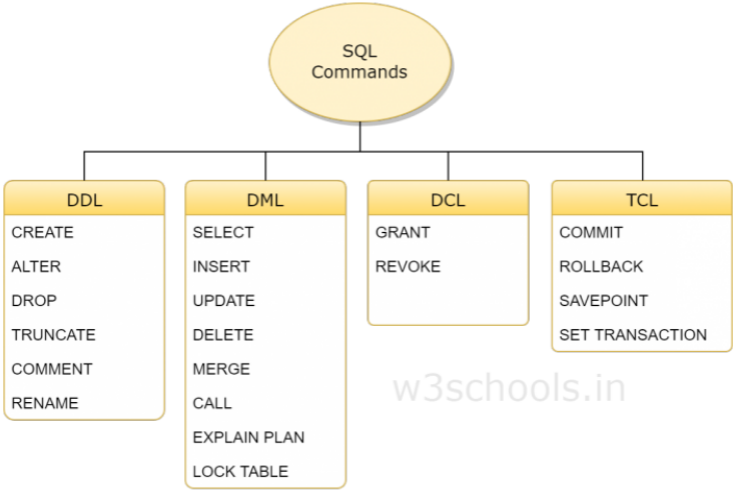
Summary

SQL DDL provides means to:

- Create, feed, modify and alter relations and views
- Specify some kinds of constraints. Some others are still under the responsability of the application programs

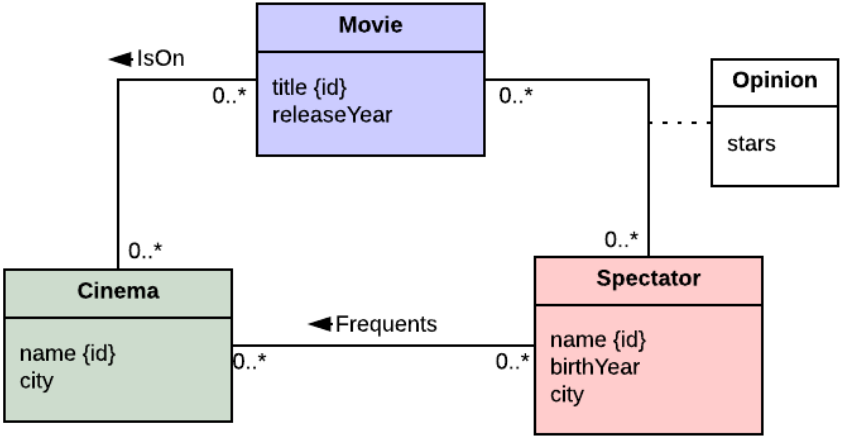
There are many RDBMS's features we didn't study: users, indexes, grant, tablespace...

SQL Commands - These slides concerns...



- DDL – Data Definition Language
- DML – Data Manipulation Language (SELECT)
- DCL – Data Control Language
- TCL - Transaction Control Language

A Toy database



Relation specifications

Toy Database Relational Model

- *Movies (title, releaseYear)*
  $< t, y > \in$ ***Movies*** $\iff$ {the movie is identified by the title ***t*** and has a release year ***y***}
- *Cinemas (name, city)*
  $< n, c > \in$ ***Cinemas*** $\iff$ {the cinema identified by its name ***n*** is located in the city ***c***}
- *Spectators (name, birthYear, city)*
  $< n, b, c > \in$ ***Spectators*** $\iff$ ...
  Domains need to be detailed.
- *Frequents (spectator, cinema)*
  $< s, c > \in$ ***Frequents*** $\iff$ ...
  Frequents[spectator] $\subseteq$ Spectators[name]
  Frequents[cinema] $\subseteq$ Cinemas[name]
- *IsOn (movie, cinema)*
  $< m, c > \in$ ***IsOn*** $\iff$ ...
  IsOn[movie] $\subseteq$ Movies[title]
  IsOn[cinema] $\subseteq$ Cinemas[name]
- *Opinions (spectator, movie, stars)*
  $< sp, m, st > \in$ ***Opinions*** $\iff$ ...
  domain(stars) = {0, 1, 2, 3, 4, 5} Opinions[spectator] $\subseteq$ Spectators[name]
  Opinions[movie] $\subseteq$ Movies[title]

Relation Values

Spectators

name	birthYear	city
Marie	1970	Sydney
Adrian	1950	Cairns
Phil	1960	Sydney
Jackie	1965	Sydney
Tom	1986	Brisbane
Alizee	1988	Alice Spring
Lauranne	1986	Amsterdam

Cinemas

name	city
Hoyts CBD	Sydney
Hoyts	Brisbane
Event Cinema Myer	Brisbane
Event Cinema	Cairns
Birch Carroll and Coyles	Brisbane
Event Cinema Red Center	Alice Spring

Opinions

spectator	movie	stars
Marie	The Inbetweeners 2	0
Adrian	The Inbetweeners 2	0
Phil	The Inbetweeners 2	2
Jackie	The Inbetweeners 2	2
Tom	The Inbetweeners 2	5

IsOn

cinema	movie
Hoyts CBD	Guardians Of The Gal
Hoyts	Guardians Of The Gal
Event Cinema Myer	Guardians Of The Gal
Event Cinema	Guardians Of The Gal
Birch Carroll and Coyles	Guardians Of The Gal

Notion of query

A query is a *declarative program* that is meant to retrieve data FROM a database.
2 modes:

- Interpreted: the result is displayed on the output
- Embedded in an other program: the result is processed by the program itself.

Notion of query

A query is a *declarative program* that is meant to retrieve data FROM a database.
2 modes:

- Interpreted: the result is displayed on the output
- Embedded in an other program: the result is processed by the program itself.

Example

```
SELECT name, city          /* projection */
FROM Spectators            /* relation joins */
WHERE city = 'Sydney';     /* selection */
```

When interacting with an DBMS (Oracle)

```
SQL> SELECT name, city
  2 FROM Spectators
  3 WHERE city = 'Sydney';

NAME     CITY
——       ——
Marie    Sydney
Phil     Sydney
Jackie   Sydney

3 rows selected
SQL>
```

29 Select and Where Statements

30 Joining relations

31 Set Operators

Projection, selection

Retrieve the names and addresses of spectators who were born before 1990

Projection, selection

Retrieve the names and addresses of spectators who were born before 1990

In SQL:

```
SELECT name, city /* projection */
FROM Spectators /* relations (one or more) */
WHERE birthYear < 1990; /* selection */
```

In *SELECT ... FROM ... WHERE P*, P is a boolean condition:

- A simple condition such as
  <att. name> <comp. op.> <att. name> or
  <att. name> <comp. op.> <lit. value>

In *SELECT ... FROM ... WHERE P*, P is a boolean condition:

- A simple condition such as
  <att. name> <comp. op.> <att. name> or
  <att. name> <comp. op.> <lit. value>
- A composed condition such as
  <cond.> <bool. op.> <cond.>
  where <cond.> is either a simple condition or a composed condition.

In *SELECT ... FROM ... WHERE P*, P is a boolean condition:

- A simple condition such as
  <att. name> <comp. op.> <att. name> or
  <att. name> <comp. op.> <lit. value>
- A composed condition such as
  <cond.> <bool. op.> <cond.>
  where <cond.> is either a simple condition or a composed condition.

Comparison operators: $=$, $\neq$, $<$, $>$, etc.
Boolean operators : NOT, AND, OR
P might contain brackets.

How do the following expressions evaluate?

- A and B or C
- A and (B or C)
- not A and A
- A and not A
- not A and C

when A=true, B=false, C=true

How do the following expressions evaluate?

- A and B or C
- A and (B or C)
- not A and A
- A and not A
- not A and C

when A=true, B=false, C=true

True, True, False, False, False

Duplicated values

Retrieve the movies whose rating is 2

According to the relational data model:
The expected result is:

MOVIE
The Inbetweeners 2
Lucy

Duplicated values

Retrieve the movies whose rating is 2

In SQL:

```
SELECT movie
FROM Opinions
WHERE stars = 2;
```

According to the relational data model:
The expected result is:

MOVIE
The Inbetweeners 2
Lucy

The actual result is:

MOVIE
The Inbetweeners 2
The Inbetweeners 2
Lucy

Eliminating Duplicates

To avoid duplicated values:

In SQL:

```
SELECT DISTINCT movie
FROM Opinions
WHERE stars = 2;
```

Eliminating Duplicates

To avoid duplicated values:

In SQL:

```
SELECT DISTINCT movie
FROM Opinions
WHERE stars = 2;
```

Needs to be carefully used:

- Think about the cost of this operation!
- Might be unecessary (ex. if attribute is key): SELECT name FROM Spectators

Star (*) Convention

The symbol * denotes a list of all attributes. Example: *Retrieve information about Spectators*

In SQL:

```
SELECT * FROM Spectators;
```

name	birthYear	city
Marie	1970	Sydney
Adrian	1950	Cairns
Phil	1960	Sydney
...	...	...

- Handy for displaying both schema and value of relations
- Not to be used embedded in a program (the schema may evolve)

Renaming Attributes in the `Select`

Renaming of attributes is implemented via the AS clause within the SELECT statement

In SQL:

```
SELECT name AS spectator, birthYear, city
FROM Spectators;
```

Has the result:

spectator	birthYear	city
Marie	1970	Sydney
Adrian	1950	Cairns
Phil	1960	Sydney
...	...	...

Ordering rows

The clause ORDER BY applies only on attributes contained in the clause select (ascending by default):

Has the result:

Adrian	Australia	5
Alizee	Crooks in Clover	5
Lauranne	Crooks in Clover	5
Marie	Crooks in Clover	5
Tom	Crooks in Clover	5
Lauranne	Edward Scissorhands	5
Adrian	I, Robot	5
Marie	I, Robot	5
Alizee	Lucy	5
Lauranne	Lucy	5
Phil	Lucy	5
Marie	Pretty Woman	5
Tom	Pretty Woman	5
Tom	The Inbetweeners 2	5

In SQL:

```
SELECT spectator, movie, stars
FROM Opinions
ORDER BY stars desc, movie asc, spectator asc;
```

29 Select and Where Statements

30 Joining relations

31 Set Operators

Inner joins

```
SELECT A1, A2, ..., An                /* ⟶ to express a projection */
FROM R1 JOIN R2 ON ... JOIN Rp ON...  /* ⟶ to express relation joins */
WHERE C;                              /* ⟶ to express a selection */
```

Inner joins

```
SELECT A1, A2, ..., An                /* ⟶ to express a projection */
FROM R1 JOIN R2 ON ... JOIN Rp ON...  /* ⟶ to express relation joins */
WHERE C;                              /* ⟶ to express a selection */
```

For each movie which is on at Hoyts, find its title and release date

Inner joins

```
SELECT A1, A2, ..., An                /* ⟶ to express a projection */
FROM R1 JOIN R2 ON ... JOIN Rp ON...  /* ⟶ to express relation joins */
WHERE C;                              /* ⟶ to express a selection */
```

For each movie which is on at Hoyts, find its title and release date

In SQL:

```
SELECT title, releaseYear
FROM Movies JOIN IsOn ON (title=movie)
WHERE cinema ='Hoyts';
/* The clause ON may contain any kind of predicate. */
```

Inner joins

```
SELECT A1, A2, ..., An                /* ⟶ to express a projection */
FROM R1 JOIN R2 ON ... JOIN Rp ON...  /* ⟶ to express relation joins */
WHERE C;                              /* ⟶ to express a selection */
```

For each movie which is on at Hoyts, find its title and release date

In SQL:

```
SELECT title, releaseYear
FROM Movies JOIN IsOn ON (title=movie)
WHERE cinema ='Hoyts';
/* The clause ON may contain any kind of predicate. */
```

To express joins between two relations (or more).

Name clashes

When the query refers more than once to the same name

For each movies which are on at Hoyts, retrieve its title and the rating gave by Adrian

```
SELECT movie, stars
FROM Opinions JOIN IsOn ON (movie = movie)
WHERE cinema = 'Hoyts' and spectator = 'Adrian';
ERROR at line 1: ORA-00918: column ambiguously defined
```

Qualified notation

In fact, full attribute names should be fully qualified by a relation name:

In SQL:

```
SELECT IsOn.movie, Opinions.stars
FROM Opinions JOIN IsOn ON (Opinions.movie = IsOn.movie)
WHERE IsOn.cinema = 'Hoyts' AND Opinions.spectator = 'Adrian';
```

- Each attribute is qualified by the relation it belongs to (easier to read)
- Necessary for attributes defined in more than one relation in the `from` statement.

Multiple citations of the same relation

Retrieve pairs of different spectators who have rated the same movie. Give also, the movie name and how much stars each one gave to this movie.
Relations are renamed in the FROM clause, qualified notation is used to refer to attributes:

In SQL:

```
SELECT O1.movie, O1.spectator, O1.stars, O2.spectator, O2.stars
FROM Opinions O1 JOIN Opinions O2
        ON (O1.movie = O2.movie and O1.spectator <> O2.spectator)
```

Multiple citations of the same relation (cont.)

O1.spectator	O1.movie	O1.stars	O2.spectator	O2.movie	O2.stars
Adrian	Lucy	4	Marie	Lucy	3
Marie	Lucy	3	Adrian	Lucy	4
...	...	...	...	...	...

- Is the clause *SELECT DISTINCT* necessary to eliminate duplicates?
- How to ensure antisymmetry? R is antisymmetric if $<X, Y> \in R \implies (<Y, X> \notin R$ or $X = Y)$. We already know that for all X, $<X, X> \notin R$

Multiple citations of the same relation (cont.)

O1.spectator	O1.movie	O1.stars	O2.spectator	O2.movie	O2.stars
Adrian	Lucy	4	Marie	Lucy	3
Marie	Lucy	3	Adrian	Lucy	4
...	...	...	...	...	...

- Is the clause *SELECT DISTINCT* necessary to eliminate duplicates?
- How to ensure antisymmetry? R is antisymmetric if $<X, Y> \in R \implies (<Y, X> \notin R$ or $X = Y)$. We already know that for all X, $<X, X> \notin R$

In SQL:

```
SELECT O1.movie, O1.spectator, O1.stars, O2.spectator, O2.stars
FROM Opinions O1 JOIN Opinions O2
        ON (O1.movie = O2.movie and O1.spectator < O2.spectator);
```

O1.movie	O1.spectator	O1.stars	O2.spectator	O2.stars
Lucy	Adrian	4	Marie	3

Natural join

The joining condition applies to attributes in common indicated inside the USING
What does the following query return?

In SQL:

```
SELECT Spectators.name, city, Cinemas.name
FROM Spectators JOIN Cinemas USING (city); /* The JOIN applies on city only */
```

A variant:

```
SELECT Spectators.name, Spectators.city, Cinemas.name
FROM Spectators JOIN Cinemas ON (Spectators.city=Cinemas.city); /* explicit join */
```

Cartesian Product

Give all the possible cinemas where Phil can go

```
SELECT *
FROM Cinemas CROSS JOIN Spectators
WHERE Spectators.name = 'Phil';
```

Joining relations: summary

Let R and S be defined as: R (X, Y, Z) and S (Y, Z, T)

- Cross join: FROM R CROSS JOIN S
  schema: R.X, R.Y, R.Z, S.Y, S.Z, S.T
- Inner Join: FROM R JOIN S ON (P)
  schema: R.X, R.Y, R.Z, S.Y, S.Z, S.T
  P is a valid predicate on R.X, R.Y, R.Z, S.Y, S.Z, S.T
  Joining condition is: P
- Natural Join: FROM R JOIN S USING (Y)
  schema: R.X, Y, R.Z, S.Z, S.T
  Joining condition is R.Y = S.Y
- Other type of JOIN exists (Outer, Self ...)

29 Select and Where Statements

30 Joining relations

31 Set Operators

Operators

- UNION [all]
  Duplicates not eliminated when all present.
- INTERSECT (stands for Intersection)
- MINUS (or EXCEPT) (stands for difference)

Results contain no duplicated values except for UNION ALL.

Minus: example

Find movies rated 5 stars which are not on at Hoyts.

In SQL:

```
SELECT movie
FROM Opinions
WHERE stars = 5
MINUS /* EXCEPT in sqlite3 */
SELECT movie
FROM IsOn
WHERE cinema = 'Hoyts';
```

Both queries must return relations whose schemas are compatible.

Minus: example

Find movies rated 5 stars which are not on at Hoyts.

In SQL:

```
SELECT movie
FROM Opinions
WHERE stars = 5
MINUS /* EXCEPT in sqlite3 */
SELECT movie
FROM IsOn
WHERE cinema = 'Hoyts';
```

Both queries must return relations whose schemas are compatible.

Intersection: example

Find the movies and spectators such that the spectator has rated the movie with more than 4 stars and frequents a cinema that has it on.

In SQL:

```
SELECT spectator, movie FROM Opinions WHERE stars ≥ 4
INTERSECT
SELECT spectator, movie FROM IsOn JOIN Frequents USING (cinema);
```

A variant:

Intersection: example

Find the movies and spectators such that the spectator has rated the movie with more than 4 stars and frequents a cinema that has it on.

In SQL:

```
SELECT spectator, movie FROM Opinions WHERE stars ≥ 4
INTERSECT
SELECT spectator, movie FROM IsOn JOIN Frequents USING (cinema);
```

A variant:

In SQL:

```
SELECT DISTINCT spectator, movie
FROM Opinions JOIN IsOn USING (movie) JOIN Frequents USING (cinema, spectator)
WHERE stars ≥4;
```

Union: example

Find the spectators who have rated "Crooks in Clover" 5 stars or frequent the Hoyts

In SQL:

```
SELECT spectator FROM Opinions
WHERE movie = 'Crooks in Clover' AND stars=5
UNION
SELECT spectator FROM Frequents
WHERE cinema = 'Hoyts';
```

$\longrightarrow$

spectator
Marie
Tom
Alizée
Lauranne
Adrian

Union: example

Find the spectators who have rated "Crooks in Clover" 5 stars or frequent the Hoyts

In SQL:

```
SELECT spectator FROM Opinions
WHERE movie = 'Crooks in Clover' AND stars=5
UNION
SELECT spectator FROM Frequents
WHERE cinema = 'Hoyts';
```

$\longrightarrow$

spectator
Marie
Tom
Alizée
Lauranne
Adrian

In SQL:

```
SELECT spectator FROM Opinions
WHERE movie = 'Crooks in Clover' AND stars=5
UNION ALL
SELECT spectator FROM Frequents
WHERE cinema = 'Hoyts';
```

$\longrightarrow$

spectator
Marie
Tom
Alizée
Lauranne
Adrian
Tom
Marie

32 Nested queries

33 Aggregation and Grouping

34 Missing Values

35 Conclusion

Queries in the WHERE clause

Principle: the FROM clause accepts SQL queries as well as relation names

For each movie rated 5 stars which is not on at Hoyts, return its name and year of release.

We know that the following query returns movies rated 5 stars which are not on at Hoyts:

In SQL:

```
SELECT movie FROM Opinions WHERE stars = 5
MINUS
SELECT movie FROM IsOn WHERE cinema = 'Hoyts';
```

Let's call R the corresponding relation.

Hence the query is (using R and the in operator):

```
SELECT title, releaseYear
FROM Movies
WHERE title IN (R)
```

Now, change R with the query that returns it:

In SQL:

```
SELECT title, releaseYear
FROM Movies
WHERE title IN (SELECT movie FROM Opinions WHERE stars = 5
                MINUS
                SELECT movie FROM IsOn WHERE cinema = 'Hoyts');
        /* < m > ∈ R ⟺ m is movie rated 5 stars which is not on at Hoyts. */
```

Sub-queries in the WHERE clause are the more readable.

IN operator

- *A query result is a relation.*
- conversely for tuple NOT IN relation

Find the name and city of the cinema Phil frequents

In SQL:

IN operator

- *A query result is a relation.*
- conversely for tuple NOT IN relation

Find the name and city of the cinema Phil frequents

In SQL:

```
(SELECT cinema FROM Frequents
WHERE spectator = 'Phil');
```

IN operator

- *A query result is a relation.*
- conversely for tuple NOT IN relation

Find the name and city of the cinema Phil frequents

In SQL:

```
SELECT name, city FROM Cinemas
WHERE name IN (SELECT cinema FROM Frequents
               WHERE spectator = 'Phil');
```

IN operator

- *A query result is a relation.*
- conversely for tuple NOT IN relation

Find the name and city of the cinema Phil frequents

In SQL:

```
SELECT name, city FROM Cinemas
WHERE name IN (SELECT cinema FROM Frequents
                    WHERE spectator = 'Phil');
```

Alternative expression:

In SQL:

```
SELECT C.name, C.city
FROM Cinemas C JOIN Frequents F ON (C.name = F.cinema)
WHERE spectator = 'Phil';
```

.

General form of IN operator

The In operator can be generalised to fit tuples of any length

- $<X_1, X_2, ..., X_n>$ IN relation (whose attributes are $A_1, A_2, ..., A_n$) is true $\iff <X_1, X_2, ...X_n> \in$ relation.

```
SELECT ... FROM ...
WHERE (X1, X2, ..., Xn) in
    (SELECT A1, A2, ..., An FROM ....)
```

Of course, for each i=1..n,
types of Xi and Ai must be comparable

Sub-queries: good practice

Give movies which have the same rating as 'Australia' given by Marie

In SQL:

```
SELECT movies /* Successfully executed when the sub-query returns one tuple only. */
FROM Opinions
WHERE stars = (SELECT stars FROM Opinions
               WHERE spectator = 'Marie' AND movie = 'Australia');
```

Sub-queries: good practice

Give movies which have the same rating as 'Australia' given by Marie

In SQL:

```
SELECT movies /* Successfully executed when the sub-query returns one tuple only. */
FROM Opinions
WHERE stars = (SELECT stars FROM Opinions
               WHERE spectator = 'Marie' AND movie = 'Australia');
```

In SQL - Better (more generic):

```
SELECT movies
FROM Opinions
WHERE stars IN (SELECT stars FROM Opinions
                WHERE spectator = 'Marie' AND movie = 'Australia');
```

Sub-queries: good practice

Give movies which have the same rating as 'Australia' given by Marie

In SQL:

```
SELECT movies /* Successfully executed when the sub-query returns one tuple only. */
FROM Opinions
WHERE stars = (SELECT stars FROM Opinions
               WHERE spectator = 'Marie' AND movie = 'Australia');
```

In SQL - Better (more generic):

```
SELECT movies
FROM Opinions
WHERE stars IN (SELECT stars FROM Opinions
                WHERE spectator = 'Marie' AND movie = 'Australia');
```

In SQL - Other option:

```
SELECT o1.movies
FROM Opinions o1 JOIN Opinions o2 USING (stars)
WHERE o2.spectator = 'Marie' AND o2.movie = 'Australia');
```

Queries in the FROM clause (not recommended)

Principle: the FROM clause accepts SQL queries as well as relation names

For each movie rated 5 stars which is not on at Hoyts, return its name and year of release.

We know that the following query returns movies rated 5 stars which are not on at Hoyts:

In SQL:

```
SELECT movie FROM Opinions WHERE stars = 5
MINUS
SELECT movie FROM IsOn WHERE cinema = 'Hoyts';
```

Let's call R the corresponding relation.

Hence the query is (using R):

In SQL:

```
SELECT title, releaseYear
FROM Movies JOIN R ON (movie = title);
```

Now, change R with the query that returns it:

In SQL:

```
SELECT title, releaseYear
FROM Movies JOIN (SELECT movie FROM Opinions WHERE stars = 5
                  MINUS
                  SELECT movie FROM IsOn WHERE cinema = 'Hoyts') R
     ON (movie = title);
```

The sub-query must be named.

WITH AS - Subquery factoring (recommended)

To introduce a temporary relation:

```
WITH R AS (
    /* a query that constructs a relation, let's call it R */
)
SELECT ... FROM R .... ;
/* Another query that uses R... */
```

In SQL:

```
WITH R AS (
    SELECT movie FROM Opinions WHERE stars = 5
    MINUS
    SELECT movie FROM IsOn WHERE cinema = 'Hoyts'
)
SELECT title, releaseYear
FROM Movies JOIN R ON (movie = title);
```

Applying subquery factoring

Give spectators who have rated all movies.

However, we know that, $|A| = |B| \wedge A \subseteq B \implies A = B$. Hence, the query is:

In SQL:

```
WITH NbRatedMoviesPerSpectator AS (
        SELECT spectator, COUNT(movie) AS nbM
        FROM Opinions
        GROUP BY spectator),
    TotalMovies as (
        SELECT COUNT(title) AS nbTot
        FROM Movies)
SELECT spectator
FROM NbRatedMoviesPerSpectator JOIN TotalMovies ON (nbM=nbTot);
```

Advantages of sub-queries

- Split the query into (simpler) queries.
- Code and test the sub-queries independently from each others.

Be careful. Sometimes a JOIN is enough.
Movies rated 5 stars, at least once:

Advantages of sub-queries

- Split the query into (simpler) queries.
- Code and test the sub-queries independently from each others.

Be careful. Sometimes a JOIN is enough.
Movies rated 5 stars, at least once:

In SQL:

```
SELECT DISTINCT title, releaseYear
FROM Opinions JOIN Movies ON (movie=title)
WHERE stars=5;
```

32 Nested queries

33 Aggregation and Grouping

34 Missing Values

35 Conclusion

Aggregation

To reduce a list of values to one value.

- COUNT (*) $\longrightarrow$ number of tuples
- COUNT (A) $\longrightarrow$ number of values in A
- COUNT (DISTINCT A) $\longrightarrow$ number of different values in A
- AVG (A) $\longrightarrow$ average value of values in A
- MIN (A) (resp. MAX) $\longrightarrow$ minimum (resp. Maximum) value of values in A
- SUM (A) $\longrightarrow$ sum value of values in A

Examples

What is the average rating given to 'Australia'?

In SQL:

Examples

What is the average rating given to 'Australia'?

In SQL:

```
SELECT      stars  FROM Opinions WHERE movie = 'Australia';
```

Examples

What is the average rating given to 'Australia'?

In SQL:

```
SELECT AVG (stars) FROM Opinions WHERE movie = 'Australia';
```

Examples

What is the average rating given to 'Australia'?

In SQL:

```
SELECT AVG (stars) FROM Opinions WHERE movie = 'Australia';
```

How many cinemas are located in Brisbane?

In SQL:

Examples

What is the average rating given to 'Australia'?

In SQL:

```
SELECT AVG (stars) FROM Opinions WHERE movie = 'Australia';
```

How many cinemas are located in Brisbane?

In SQL:

```
SELECT * FROM Cinemas WHERE city = 'Brisbane';
```

Examples

What is the average rating given to 'Australia'?

In SQL:

```
SELECT AVG (stars) FROM Opinions WHERE movie = 'Australia';
```

How many cinemas are located in Brisbane?

In SQL:

```
SELECT COUNT (*) FROM Cinemas WHERE city = 'Brisbane';
```

How many movies are rated more than 2?

In SQL:

SELECT COUNT (movie) FROM Opinions WHERE stars > 2;

How many movies are rated more than 2?

In SQL:

SELECT COUNT (movie) FROM Opinions WHERE stars $>$ 2;

The query above is incorrect, why?

How many movies are rated more than 2?

In SQL:

```
SELECT COUNT (movie) FROM Opinions WHERE stars > 2;
```

The query above is incorrect, why?

It counts a movie as many times there exists a spectator who rated it more than 2 stars

How many movies are rated more than 2?

In SQL:

```
SELECT COUNT (movie) FROM Opinions WHERE stars > 2;
```

The query above is incorrect, why?
It counts a movie as many times there exists a spectator who rated it more than 2 stars
A correct expression is:

In SQL:

```
SELECT COUNT (DISTINCT movie) FROM Opinions WHERE stars > 2;
```

Give names of spectators who rated the movie Australia with the less stars

In SQL:

Give names of spectators who rated the movie Australia with the less stars

In SQL:

```
SELECT spectator
FROM Opinions
WHERE movie='Australia' AND stars IN
        (SELECT MIN(stars) FROM Opinions
        WHERE movie='Australia');
```

Grouping

Partition a relation to apply an aggregation on each class separately

Consider the query: *How many cinemas does each spectator frequent?*

Expected result:

spectator	**cinema**
Adrian	2
Alizée	1
Jackie	2
Marie	2
Phil	2
Tom	2

Processing a grouping

1. Build the partition according to the spectator value

Adrian	Hoyts CBD
Adrian	Hoyts
Alizee	Event Cinema
Jackie	Hoyts CBD
Jackie	Event Cinema
Marie	Hoyts
Marie	Hoyts CBD
Phil	Event Cinema
Phil	Hoyts CBD
Tom	Hoyts
Tom	Birch Carroll and Coyles

In SQL:

```
SELECT ...
FROM Frequents
GROUP BY spectator;
```

2. Reduce to one tuple only each class of this partition

list of spectator (partition criteria)
$\longrightarrow$ spectator (one of the values)
list of cinema
$\longrightarrow$ integer (the number of values)

In SQL:

```
SELECT spectator, COUNT (cinema)
FROM Frequents
GROUP BY spectator;
```

Impact of partitioning on the SELECT clause

In queries that contain the GROUP BY clause, the SELECT clause contains only:

- One or more attributes among those present in the partition criteria
- One or more aggregations which apply on other attributes

How about, if we need to add an attribute in the SELECT clause to answer the query?

Incorrect query:

```
SELECT spectator, city, COUNT(cinema)
FROM Frequents JOIN Spectators ON (spectator=name)
GROUP BY spectator;
```

Correct query:

```
SELECT spectator, city, COUNT(cinema)
FROM Frequents JOIN Spectators ON (spectator=name)
GROUP BY spectator, city;
```

Filtering a partition

The clause HAVING, which comes with the clause GROUP BY is meant to filter classes resulting from the partition.
Rules on SELECT clause (with grouping) apply on HAVING as well.
For each spectator who has rated more than 2 movies, retrieve the Maximum rating he/she gave and the number of cinema he/she frequents.

In SQL:

;

Filtering a partition

The clause HAVING, which comes with the clause GROUP BY is meant to filter classes resulting from the partition.
Rules on SELECT clause (with grouping) apply on HAVING as well.
For each spectator who has rated more than 2 movies, retrieve the Maximum rating he/she gave and the number of cinema he/she frequents.

In SQL:

```
FROM Opinions JOIN Frequents USING (spectator)

                                                ;
```

Filtering a partition

The clause HAVING, which comes with the clause GROUP BY is meant to filter classes resulting from the partition.
Rules on SELECT clause (with grouping) apply on HAVING as well.
For each spectator who has rated more than 2 movies, retrieve the Maximum rating he/she gave and the number of cinema he/she frequents.

In SQL:

```
FROM Opinions JOIN Frequents USING (spectator)
GROUP BY spectator
                                                    ;
```

Filtering a partition

The clause HAVING, which comes with the clause GROUP BY is meant to filter classes resulting from the partition.
Rules on SELECT clause (with grouping) apply on HAVING as well.
For each spectator who has rated more than 2 movies, retrieve the Maximum rating he/she gave and the number of cinema he/she frequents.

In SQL:

```
FROM Opinions JOIN Frequents USING (spectator)
GROUP BY spectator
HAVING COUNT (DISTINCT movie) > 2;
```

Filtering a partition

The clause HAVING, which comes with the clause GROUP BY is meant to filter classes resulting from the partition.
Rules on SELECT clause (with grouping) apply on HAVING as well.
For each spectator who has rated more than 2 movies, retrieve the Maximum rating he/she gave and the number of cinema he/she frequents.

In SQL:

```
SELECT spectator, MAX(stars) as maxRating, COUNT (DISTINCT cinema) as nbCines
FROM Opinions JOIN Frequents USING (spectator)
GROUP BY spectator
HAVING COUNT (DISTINCT movie) > 2;
```

Query operational semantics

FROM *1. cartesian product/JOIN*

Query operational semantics

FROM *1. cartesian product/JOIN*
WHERE *2. Selection on input relations*

Query operational semantics

FROM *1. cartesian product/JOIN*
WHERE *2. Selection on input relations*
GROUP BY *3. partition*

Query operational semantics

FROM *1. cartesian product/JOIN*
WHERE *2. Selection on input relations*
GROUP BY *3. partition*
HAVING *4. filtering the partition*

Query operational semantics

SELECT *5. projection on partition criteria and aggregations*
FROM *1. cartesian product/JOIN*
WHERE *2. Selection on input relations*
GROUP BY *3. partition*
HAVING *4. filtering the partition*

Grouping: common mistake

For each cinema retrieve its name and city, the number of movies which are on and the spectators who frequent it.

```
SELECT cinema, COUNT (DISTINCT movie) as nbM, city, spectator
                                                *
FROM Cinemas JOIN IsOn ON (name = cinema) JOIN Frequents USING (cinema)
GROUP BY cinema;
ERROR at line 1:
ORA-00979: not a GROUP BY expression
```

Why?

Expected result

cinema	**city**	**nbM**	**spectator**
Hoyts CBD	Sydney	3	{Adrian, Jackie, Marie, Phil}
Hoyts	Brisbane	3	{Tom, Marie, Adrian}
Event Cinema	Cairns	5	{Alizée, Phil, Jackie}
Birch Carroll and Coyles	Brisbane	5	{Tom}

The type of attribut spectator is a set.
Sets are not domains of atomic values!

Possible result

cinema	city	nbM	spectator
Hoyts CBD	Sydney	3	Adrian
Hoyts CBD	Sydney	3	Jackie
Hoyts CBD	Sydney	3	Marie
Hoyts CBD	Sydney	3	Phil
Hoyts	Brisbane	3	Tom
Hoyts	Brisbane	3	Marie
Hoyts	Brisbane	3	Adrian
Event Cinema	Cairns	5	Alizée
Event Cinema	Cairns	5	Phil
Event Cinema	Cairns	5	Jackie
Birch Carroll and Coyles	Brisbane	5	Tom

How about in Mysql or SQLite?

Mysql (Ver 14.14 Distrib 5.5.19, for osx10.6 (i386)) or SQlite (v 3) returns:

name	**city**	**nbM**	**spectator**
Hoyts CBD	Sydney	3	Adrian
Hoyts	Brisbane	3	Tom
Event Cinema	Cairns	5	Alizée
Birch Carroll and Coyles	Brisbane	5	Tom

which is incorrect....

To fix city problem

In SQL:

SELECT cinema, COUNT(movie) as nbM, **city**
FROM Cinema JOIN IsOn ON (name=cinema)
GROUP BY cinema, **city**;

Both expressions
GROUP BY cinema and *GROUP BY cinema, city*
build the same partition because each cinema has only one city.

To fix spectator problem

In SQL:

```
SELECT cinema, nbM, city, spectator
FROM (SELECT cinema, COUNT(movie) AS nbM, city
      FROM Cinemas JOIN IsOn ON (name=cinema)
      GROUP BY cinema, city) X
      /* <c, n, a> ∈ X ⟺ the cinema c has n movies on and is located in city a. */
      JOIN Frequents USING (cinema);
```

To fix spectator problem

In SQL:

```
SELECT cinema, nbM, city, spectator
FROM (SELECT cinema, COUNT(movie) AS nbM, city
        FROM Cinemas JOIN IsOn ON (name=cinema)
        GROUP BY cinema, city) X
        /* <c, n, a> ∈ X ⟺ the cinema c has n movies on and is located in city a. */
        JOIN Frequents USING (cinema);
```

In SQL - Better option:

```
WITH NbMoviesPerCinema AS (
    SELECT cinema, COUNT(movie) as nbM, city
    FROM Cinemas JOIN IsOn ON (name=cinema)
    GROUP BY cinema, city)
SELECT cinema, nbM, city, spectator
FROM NbMoviesPerCinema JOIN Frequents USING (cinema);
```

Exercice

- *Give the cinemas which are frequented by the most spectators.*
- *For each spectator who has rated all movies on at Pathé, return his name and the cinemas he/she frequents.*

Opinions (spectator, movie, stars)

spectator	movie	stars
Marie	Dune 2	4
Adrian	Dune 2	5
Phil	Dune 2	5
Marie	Le livre des solutions	3
Adrian	Le livre des solutions	4
Phil	Anatomie d'une chute	4
Adrian	Anatomie d'une chute	4

Frequents (spectator, cinema)

spectator	cinema
Marie	Pathé
Adrian	Pathé
Phil	Le Club
Marie	Le Club
Adrian	Le Club
Marie	La Nef

IsOn (movie, cinema)

movie	cinema
Dune 2	Pathé
Le livre des solutions	Pathé
Dune 2	Le Club
Anatomie d'une chute	Le Club
Dune 2	La Nef

Give the cinemas which are frequented by the most spectators.

In SQL:

```
WITH NbSpecPerCin AS (SELECT cinema, COUNT(spectator) as nbS
                      FROM Frequents
                      GROUP BY cinema),
     MaxSpecCin AS (SELECT MAX(nbS) as maxS
                      FROM NbSpecPerCin)
SELECT cinema
FROM NbSpecPerCin JOIN MaxSpecCin ON (nbS = maxS) ;
```

In SQL (Oracle - Not SQLite):

```
SELECT cinema
FROM Frequents
GROUP BY cinema
HAVING COUNT(spectator)= (SELECT MAX(COUNT(spectator))
                      FROM Frequents
                      GROUP BY cinema);
```

For each spectator who has rated all movies on at Pathé, return his name and the cinemas he/she frequents.

In SQL:

```
WITH NbMoviesPathe AS (
        SELECT COUNT(movie) AS nbT
        FROM IsOn
        WHERE cinema = 'Pathé' ),
   RatedMoviesPathePerSpec AS (
        SELECT spectator, COUNT(movie) as nbM
        FROM Opinions JOIN IsOn USING (movie)
        WHERE cinema = 'Pathé'
        GROUP BY spectator)
SELECT spectator, cinema
FROM NbMoviesPathe JOIN RatedMoviesPathePerSpec ON (nbT=nbM)
                   JOIN Frequents USING (spectator);
```

32 Nested queries

33 Aggregation and Grouping

34 Missing Values

35 Conclusion

Motivation

Sometimes we need missing values:

- Value temporarily unknown:
  I don't know his address now, but I can find it soon
- Value temporarily forbidden:
  She is not married, so she can't have a marital name
- Value definitively forbidden:
  He didn't attend this exam, so he cannot get a mark for it.
- etc.

NULL value

- The missing value is denoted *NULL*.
- In outputs, NULL values are not displayed (nothing appears).
- NULL belongs to all types.
- All operators are revisited.
- Two new operators: IS NULL - IS NOT NULL

Operators with null values

- Comparison operators: let A be an attribute.
  $\forall$ op $\in \{<, >, <>, =\}$, A op Null, Null op A are Null

Operators with null values

- Comparison operators: let A be an attribute.
  $\forall$ op $\in \{<, >, <>, =\}$, A op Null, Null op A are Null
- Arithmetic operators:
  $\forall$ op $\in$ {+, -, /, *, ...}, A op Null, Null op A are Null

Operators with null values

- Comparison operators: let A be an attribute.
  $\forall$ op $\in \{<, >, <>, =\}$, A op Null, Null op A are Null
- Arithmetic operators:
  $\forall$ op $\in$ {+, -, /, *, ...}, A op Null, Null op A are Null
- Logical operators:

AND	True	False	Null
True	True	False	Null
False	False	False	False
Null	Null	False	Null

Operators with null values

- Comparison operators: let A be an attribute.
  $\forall$ op $\in \{<, >, <>, =\}$, A op Null, Null op A are Null
- Arithmetic operators:
  $\forall$ op $\in$ {+, -, /, *, ...}, A op Null, Null op A are Null
- Logical operators:

AND	True	False	Null
True	True	False	Null
False	False	False	False
Null	Null	False	Null

OR	True	False	Null
True	True	True	True
False	True	False	Null
Null	True	Null	Null

Operators with null values

- Comparison operators: let A be an attribute.
  $\forall$ op $\in \{<, >, <>, =\}$, A op Null, Null op A are Null
- Arithmetic operators:
  $\forall$ op $\in$ {+, -, /, *, ...}, A op Null, Null op A are Null
- Logical operators:

AND	True	False	Null
True	True	False	Null
False	False	False	False
Null	Null	False	Null

Similarly NOT null is null

OR	True	False	Null
True	True	True	True
False	True	False	Null
Null	True	Null	Null

- MIN, MAX, SUM, AVG ignore null values.
  If all values are null, then the result is null.

- MIN, MAX, SUM, AVG ignore null values.
  If all values are null, then the result is null.
- COUNT(*) doesn't care about null values: it counts tuples.

- MIN, MAX, SUM, AVG ignore null values.
  If all values are null, then the result is null.
- COUNT(*) doesn't care about null values: it counts tuples.
- COUNT (DISTINCT A) and COUNT (A) count only non null values

- MIN, MAX, SUM, AVG ignore null values.
  If all values are null, then the result is null.
- COUNT(*) doesn't care about null values: it counts tuples.
- COUNT (DISTINCT A) and COUNT (A) count only non null values

NULL is neither 0, nor { }, nor '', nor ()
NULL is nothing!

Queries with null values

R	**idStudent**	**Mark**	**Course**
	12	45	math
	11	90	french
	11	75	math
	12	65	physics
	12		french

Find information about students who have got a mark different than 90

In SQL:

```
SELECT * FROM R WHERE mark <> 90;
```

Find information about students who have got a mark different than 90

In SQL:

```
SELECT * FROM R WHERE mark <> 90;
```

R	**idStudent**	**Mark**	**Course**
	12	45	math
	11	75	math
	12	65	physics

Find information about students who have got a mark different than 90

In SQL:

```
SELECT * FROM R WHERE mark <> 90;
```

R	**idStudent**	**Mark**	**Course**
	12	45	math
	11	75	math
	12	65	physics

A tuple is selected if it satisfies the where condition
(if the condition evaluates to TRUE)

Find information about students who have got a mark different than 90 (including those who didn't attend)

In SQL:

```
SELECT * FROM R WHERE mark <> 90 OR mark IS NULL;
```

R	**idStudent**	**Mark**	**Course**
	12	45	math
	11	75	math
	12	65	physics
	12		french

For each student, compute its average mark.

For each student, compute its average mark.
Unattented exam counts for 0

In SQL:

```
SELECT idStudent, AVG (NVL (mark, 0)) as avgMark
FROM R GROUP BY idStudent;
    /* NVL (x, y): if x is not null then x else y */
```

R	**idStudent**	**AvgMark**
	12	36.67
	11	82.5

Generating null values

Spectators

name	birthYear	city
Adrian	1950	Cairns
Marie	1970	Sydney
Lauranne	1986	Amsterdam

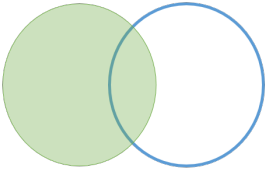
Frequents

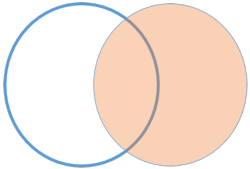
spectator	cinema
Adrian	Hoyts CBD
Adrian	Hoyts
Marie	Hoyts CBD

Cinemas

name	city
Hoyts CBD	Sydney
Hoyts	Brisbane
Event Cinema Red Center	Alice Spring

```
SELECT *
FROM Spectators
  LEFT OUTER JOIN
     Frequents ON
  name=spectator
```





```
SELECT *
FROM Frequents
  RIGHT OUTER JOIN
     Cinemas ON
  cinema=name
```

name	birthYear	city	spectator	cinema
Adrian	1950	Cairns	Adrian	Hoyts CBD
Adrian	1950	Cairns	Adrian	Hoyts
Marie	1970	Sydney	Marie	Hoyts CBD
Lauranne	1986	Amsterdam	NULL	NULL

spectator	cinema	name	city
Adrian	Hoyts CBD	Hoyts CBD	Sydney
Adrian	Hoyts	Hoyts	Brisbane
Marie	Hoyts CBD	Hoyts CBD	Sydney
NULL	NULL	Event Cinema Red Center	Alice Spring

For each cinema, find the spectators who frequent it (including where nobody goes)

In SQL:

```
SELECT F.spectator, C.name
FROM Cinemas C LEFT OUTER JOIN Frequents F ON (C.name = F.cinema);
```

For each row in Cinemas that does not satisfy the join condition with any row in Frequents, a joined row is returned with NULL values in columns of Frequents.

spectator	cinema
Marie	Hoyts CBD
Adrian	Hoyts CBD
Phil	Hoyts CBD
Jackie	Hoyts CBD
Tom	Hoyts
Alizee	Event Cinema
Marie	Hoyts
Adrian	Hoyts
Phil	Event Cinema
Jackie	Event Cinema
Tom	Birch Carroll and Coyles

What is the result of?:

```
SELECT F.spectator, F.cinema
FROM Cinemas C LEFT OUTER JOIN
        Frequents F ON (C.name = F.cinema);
```

Attributes in the projection need to be carefully chosen!

Another example :

Students (<u>stuId</u>, firstName, lastName)
Courses (<u>courseId</u>, name)
Enrolls (<u>*stuId, courseId*</u>)

For each student (stuId, firstName and lastName) find course ids in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

In SQL:

Another example :

Students (stuId, firstName, lastName)
Courses (courseId, name)
Enrolls (*stuId, courseId*)

For each student (stuId, firstName and lastName) find course ids in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

In SQL:

```
SELECT Students.stuId, firstName, lastName
FROM Students LEFT OUTER JOIN Enrolls ON (Students.stuId=Enrolls.stuid);
```

For each row in *Students* that does not satisfy the join condition with any row in Enrolls, a joined row is returned with NULL values in columns of *Enrolls*.

Another example :

Students (<u>stuId</u>, firstName, lastName)
Courses (<u>courseId</u>, name)
Enrolls (<u>*stuId, courseId*</u>)

For each student (stuId, firstName and lastName) find course ids in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

In SQL:

```
SELECT Students.stuId, firstName, lastName, courseId
FROM Students LEFT OUTER JOIN Enrolls ON (Students.stuId=Enrolls.stuid);
```

For each row in *Students* that does not satisfy the join condition with any row in Enrolls, a joined row is returned with NULL values in columns of *Enrolls*.

Yet another example:

For each student (stuId, firstName and lastName) find the id and name of the courses in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

Yet another example:
For each student (stuId, firstName and lastName) find the id and name of the courses in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

In SQL:

```
FROM Students S LEFT OUTER JOIN Enrolls E ON (S.stuId=E.stuId)
    LEFT OUTER JOIN Courses C ON (E.courseId=C.courseId);
```

Yet another example:
For each student (stuId, firstName and lastName) find the id and name of the courses in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

In SQL:

```
SELECT S.stuId, firstName, lastName, C.courseId, name
FROM Students S LEFT OUTER JOIN Enrolls E ON (S.stuId=E.stuId)
    LEFT OUTER JOIN Courses C ON (E.courseId=C.courseId);
```

Yet another example:
For each student (stuId, firstName and lastName) find the id and name of the courses in which she/he is enrolled (retrieve also students who aren't enrolled in any course)

In SQL:

```
SELECT S.stuId, firstName, lastName, C.courseId, name
FROM Students S LEFT OUTER JOIN Enrolls E ON (S.stuId=E.stuId)
    LEFT OUTER JOIN Courses C ON (E.courseId=C.courseId);
```

The following query returns the same result:

In SQL:

```
SELECT S.stuId, firstName, lastName, C.courseId, name
FROM Courses C RIGHT OUTER JOIN Enrolls E ON (C.courseId=E.courseId)
    RIGHT OUTER JOIN Students S ON (E.stuId=S.stuId);
```

Some DBMS such as SQLite does not support RIGHT OUTER JOIN

32 Nested queries

33 Aggregation and Grouping

34 Missing Values

35 Conclusion

Summary

- We studied a part of SQL Data Manipulation Language only (Data Definition Language still to be seen)
- Many ways to express a given query: how to choose?
  - Not that simple...
  - As queries are optimised by the DBMS, this is not a matter the programmer has to worry about..
  - However, depending on the DBMS, query evaluation response time could be en issue.

Chapter 10 – Normalization

36 Motivations

37 Design Anomalies

38 Functional Dependencies

39 Normalisation

40 Conclusion

As noted earlier, the relational model is Simple, uniform, well-defined, formal, ... Such properties lead to useful mathematical theories: such as functional dependency (fd) and normalisation.

The basis for yet another approach to relational design:

- *Bottom-up*: unstructured schema, then normalisation (via fds)

Contrast this to the other approach:

- *Top-down*: conceptual (UML CD) design, then conceptual-to-relational mapping

Essentially, functional dependencies

- are a kind of constraint among attributes within a relation (i.e., spec)
- have implications for “goodnes” of relational schema designs

What we study here:

- basic theory and definition of functional dependencies
- methodology for assessing and improving schema design (normalisation)

Chapter 10 – Normalization

36 Motivations

37 Design Anomalies

38 Functional Dependencies

39 Normalisation

40 Conclusion

Relational Design and Redundancy

A good relational database design:

- Must capture all of the necessary attributes/associations
- Should do so with a minimal data redundancy.
  Redundancy leads to difficulties with maintaining consistency during updates, insertions and deletions

Consider the following relation (Accounts) defining 2 persons with different accounts. We suppose that there cannot be shared accounts.
It shows some redundancies...:

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	La Nef	Paris	400000
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

Consider the following relation (Accounts) defining 2 persons with different accounts. We suppose that there cannot be shared accounts.
It shows some redundancies...:

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	La Nef	Paris	400000
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

Insertion anomaly: insert account 301 for client 2222222 in Credit Cooperatif.
We need to check that bank data is consistent with existing tuples

customer	accNo	balance	bank	address	assets
1313131	101	600	*Credit Cooperatif*	*Paris*	*9000000*
1313131	102	400	*Credit Cooperatif*	*Paris*	*9000000*
1313131	101	100	La Nef	Paris	400000
1111111	201	500	*Credit Cooperatif*	*Paris*	*9000000*
1111111	204	900	*Credit Cooperatif*	*Paris*	*9000000*
2222222	**301**	**0**	**Credit Cooperatif**	**Paris**	**9000000**

Update anomaly: update Credit Cooperatif bank address to Grenoble
If a bank's address changes, we need to update all tuples referring to that bank

customer	accNo	balance	bank	address	assets
1313131	101	600	*Credit Cooperatif*	*Paris*	9000000
1313131	102	400	*Credit Cooperatif*	*Paris*	9000000
1313131	101	100	La Nef	Paris	400000
1111111	201	500	*Credit Cooperatif*	*Paris*	9000000
1111111	204	900	**Credit Cooperatif**	**Grenoble**	9000000

Deletion anomaly: remove account 101 of Nef's bank

If we remove information about the last account at a bank, all the bank information disappears.

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	*La Nef*	*Paris*	*400000*
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

Summary

- To avoid these kinds of design problems: decompose the relation.
- Each relation in the decomposition is about one concept (bank, account, ...)

This is actually the purpose of a conceptual design.

So, why do we need a dependency theory and normalisation procedure to deal with redundancy?

- Normalisation can be viewed as (semi)automated design:
  - determine all of the attributes in the problem domain
  - collect them all together in a *super-relation*
  - provide some simple information about how attributes are related
  - apply normalisation to decompose into non-redundant relations

Normalisation can also be seen as a tool for relation quality assessment

Chapter 10 – Normalization

36 Motivations

37 Design Anomalies

38 Functional Dependencies

39 Normalisation

40 Conclusion

Notations & Terminology

Most texts adopt the following terminology:

- **Relation schemas**: upper-case letters, denoting set of all attributes (e.g. R, S, P, Q, ...)
- **Tuples**: lower-case letters (e.g. t, t', t1, u, v, ...)
- **Attributes**: upper-case letters from start of alphabet (e.g. A, B, C, D, ...)
- **Sets of attributes**: simple list of attribute names (e.g. X = ABCD rather than X = A,B,C,D)
- **Tuple component values**: tuple[attrSet]
  (e.g. t[X], t[Y,Z])

Definition

A functional dependency between X and Y in R,
denoted $X \longrightarrow Y$,
specifies a constraint over R:

- $\forall$ t, u $\in$ R, t[X] = u[X] $\Longrightarrow$ t[Y] = u[Y]

In other words, **if two tuples in R agree in their values for the set of attributes X, then they must also agree in their values for the set of attributes Y**.

- We say that "Y is functionally dependent on X".
- The attribute sets X and Y may overlap, and it is trivially true that $X \longrightarrow X$.

It's worth noting:

- The single arrow $\longrightarrow$ denotes *functional dependency*
- $X \longrightarrow Y$ can also be read as *X determines Y*
- The double arrow $\Longrightarrow$ denotes *logical implication*

Examples

What functional dependencies does Accounts satisfies?

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	La Nef	Paris	400000
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

Examples

What functional dependencies does Accounts satisfies?

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	La Nef	Paris	400000
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

- bank, accNo $\longrightarrow$ balance, customer
- bank $\longrightarrow$ address, assets

Examples

What functional dependencies does Accounts satisfies?

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	La Nef	Paris	400000
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

- bank, accNo $\longrightarrow$ balance, customer
- bank $\longrightarrow$ address, assets

But:

- accNo $\not\longrightarrow$ customer
- customer $\not\longrightarrow$ accNo
- customer $\not\longrightarrow$ bank
- bank $\not\longrightarrow$ customer

Inference rules

These rules (6) are known as Amstrong's axioms (1974). They are use to infer functional dependencies.

- *R1 : reflexive rule*
  if $X \subseteq Y$ then $Y \longrightarrow X$
- *R2 : augmentation rule*
  if $X \longrightarrow Y$ then $XZ \longrightarrow YZ$
- *R3 : transitive rule*
  if $X \longrightarrow Y$ and $Y \longrightarrow Z$ then $X \longrightarrow Z$
- *R4 : decomposition rule*
  if $X \longrightarrow Y$ and $Z \subseteq Y$ then $X \longrightarrow Z$
- *R5 : union rule*
  if $X \longrightarrow Y$ and $X \longrightarrow Z$ then $X \longrightarrow YZ$
- *R6 : pseudo-transitive rule*
  if $X \longrightarrow Y$ and $YW \longrightarrow Z$ then $XW \longrightarrow Z$

Examples

In the relation Accounts, a set F of FDs is:

(1) accNo, bank $\longrightarrow$ balance
(2) accNo, bank $\longrightarrow$ customer
(3) bank $\longrightarrow$ address
(4) bank $\longrightarrow$ assets

From F we can derive (among other FDs):

(5) accNo, bank $\longrightarrow$ balance, customer, address, assets
(by R2 (augmentation rule) and R5 (union rule) from 1, 2, 3 and 4)

Closure (*Fermeture*) of X under F

Definition: closure of X under F (noted X^+)

- X is set of attributes in a relation R,
- F is a set of functional dependencies which hold in R
- X^+ is the set of attributes that are functionally determined by X based on F.

Example:
{bank} closure under the set of dependencies which hold in Accounts

- $\{bank\}^+$ = {bank, address, assets}

{accNo, bank} closure under the set of dependencies which hold in Accounts

- $\{accNo, bank\}^+$ = {accNo, bank, balance, customer, address, assets}

Key of a relation

- **Key** let's U be the set of R's attributes. X ($X \subseteq U$) is key in R, $\iff X \longrightarrow U$, which means that $\{X\}^+ = U$
- **Full functional dependency (FFD)**:
  $X \longrightarrow Y$ is FFD $\iff \forall X' \subset X, X' \not\longrightarrow Y$
  (Y is fully functionally dependent by X)
- R can have more than one key.
- **Prime attribute**: Z is a prime attribute of R if it is a member of some key of R.
- **Non-prime attribute**: Z is non-prime attribute of R, if it is not a member of any key.

Example

- *accNo, bank* $\longrightarrow$ *customer* is a *Full Functional Dependency*
- *accNo, bank* $\longrightarrow$ *address* is not a *Full Functional Dependency* because we have *bank* $\longrightarrow$ *address*
- accNo, bank is a *key* in the relation because:
  accNo, bank $\longrightarrow$ customer, balance, address, assets (the closure $\{customer, accNo\}^+$ under F contains all the attributes in the relation Accounts)
- both accNo and bank are *prime attributes* as they are part of the candidate key
- balance, customer, address and assets are *non-prime attributes*

TIP: if an attribute does not appear on the right side of any functional dependency, that attribute will be part of any key of the relation.

Exercise

The following relation (self explaining) concerns employees that are in different departments and buildings.

EMPLOYEES (NumE, Name, Wage, Department, Building)

Knowing that an employee works in a given department, and that no department has premises in several buildings, find the functional dependencies and the corresponding keys.

Exercise

EMPLOYEES (NumE, Name, Wage, Department, Building)
We have NumE which unique for each employee. So:
NumE $\longrightarrow$ **Name, Wage**

We also have
NumE $\longrightarrow$ **Department** (an employee works in a department)
Departement $\longrightarrow$ **Building** (a department cannot be in several buildings)

So NumE is a relation key. (NumE $\longrightarrow$ Building, transitivity).
$\{NumE\}^+$ = {NumE, Name, Wage, Department, Building}

Chapter 10 – Normalization

36 Motivations

37 Design Anomalies

38 Functional Dependencies

39 Normalisation

40 Conclusion

Aims

- Characterise the level of redundancy in a relational schema
- Provide mechanisms for transforming schemas to remove redundancy

Normalisation relies on functional dependencies.

Normalisation process

Normalisation theory defines six normal forms (NFs).

- Each normal form:
  - involves a set of dependency properties that a schema must satisfy
  - gives guarantees about presence/absence of update anomalies
- The normalisation process:
  - check whether a relation schema is in a particular normal form (xNF)
  - if not, partition into sub-relations where each is closer to xNF
  - repeat until the desired xNF is achieved over all sub-relations

Normal forms: brief history

- First, Second, Third Normal Forms (1NF, 2NF, 3NF) (Codd 1972)
- Boyce-Codd (Kent) Normal Form (BCNF) (1974)
- Fourth Normal Form (4NF) (Zaniolo 1976, Fagin 1977)
- Fifth Normal Form (5NF) (Fagin 1979)

Normal forms are linked to each other

- 5NF $\Longrightarrow$ 4NF $\Longrightarrow$ BCNF $\Longrightarrow$ 3NF $\Longrightarrow$ 2NF $\Longrightarrow$ 1NF
- 1NF permits the most redundancy
- 5NF permits the least redundancy.

First Normal Form

A relation R is in first normal form $\iff$ *:*

- Its attribute domains are all atomic (i.e., follows the relational model)

In fact, 1NF is part of the formal definition of a relation

Bar	*nbBeers*	*Addr*	*Drinker*
Australia Hotel	1	The Rocks	{John}
Coogee Bay Hotel	4	Coogee	{Adam, John}
Lord Nelson	2	The Rocks	{Gernot, John}
Marble Bar	3	Sydney	{Justin}
Regent Hotel	2	Kingsford	{Justin}

This table is not a relation in first normal form, while Accounts is in 1NF

Second Normal Form

A relation R is in second normal form (2NF) $\iff$ *:*

- R is in 1NF
- Every non prime attribute A in R is fully functionally dependent on every key of R.

The relation Accounts is not in 2FN because:

- address and assets are not fully functionally dependent on the key {accNo, bank} as we have : bank $\longrightarrow$ address, assets

TIP: If all the keys are atomic, then the relation is in 2NF

Third Normal Form

A relation R is in third normal form (3NF) $\iff$ *:*

- R is in 2NF
- Every non prime attribute A in R is non transitively dependent on every key of R.

Third Normal Form (cont.)

We need one more definition:

- An attribute A is *transitively dependent* on X in R $\iff$:

$$\exists\, Y \not\subseteq X \begin{cases} A \not\subseteq Y \\ X \longrightarrow Y \\ Y \longrightarrow A \\ Y \not\longrightarrow X \end{cases}$$

Exercice (cont.)

Which is the normal form of the following relation?

EMPLOYEES (NumE, Name, Wage, Department, Building)

NumE $\longrightarrow$ Name, Wage, Department
Department $\longrightarrow$ Building

Exercice (cont.)

Which is the normal form of the following relation?

EMPLOYEES (NumE, Name, Wage, Department, Building)

NumE $\longrightarrow$ Name, Wage, Department
Department $\longrightarrow$ Building

The relation is in **3NF** because the key is not composite, all the FDs are total. But Building is transitively dependent on NumE.

Example

If we have:
Accounts (customer, accNo, balance, bank)
with the following FDs which hold in Accounts:

- bank, accNo $\longrightarrow$ customer, balance

Banks (bank, address, assets)
with the following FDs which hold in Accounts:

- bank $\longrightarrow$ address, assets

Are both relations in 3NF?

Example

If we have:
Accounts (customer, accNo, balance, bank)
with the following FDs which hold in Accounts:

- bank, accNo $\longrightarrow$ customer, balance

Banks (bank, address, assets)
with the following FDs which hold in Accounts:

- bank $\longrightarrow$ address, assets

Are both relations in 3NF? *Yes, because none of the non primary attributes in both relations are transitively dependent on the respective keys, which are {bank, accNo} for the Accounts relation and {bank} in the Banks relation.*

Another Example

Example: Tutors (student, course, tutor).
Functional dependencies which hold in Tutors are:

- student, course $\longrightarrow$ tutor
- tutor $\longrightarrow$ course

Another Example (cont.)

Thus, *student* has to be part of the key as it does not appear on the right hand side of the DFs (there is no attribute that determines *student* in the Tutors' relation).
We calculate the closures (applying the Amstrong's axions):

- $\{\text{student, course}\}^+ = \{\text{student, course, tutor}\}$
- $\{\text{student, tutor }\}^+ = \{\text{student, course, tutor}\}$

Hence:

- {student, course} and {tutor, student} are keys.
- Tutors is in 3FN (no non-prime attributes)

Another Example (cont.)

Unfortunately, Tutors still has redundancy:

student	course	tutor
Peter	database	John
Peter	french	Mary
Paul	database	John
Mary	database	Alan
Paul	french	Mary

the association (tutor, course) is repeated as many times as there are students enrolled in the course.

Boyce-Codd Normal Form (BCNF)

A relation R is in Boyce-Codd normal form $\iff$ *:*

- R is in 3NF
- Whenever X $\longrightarrow$ A holds in R and X $\longrightarrow$ A non-trivial, X is a key of R.

Tutors is not in BCNF because tutor $\longrightarrow$ course holds in Tutors and the left-hand side does not contain any key.

Normal forms: summary

- **1NF** all attributes have atomic values
  we assume this as part of relational model
- **2NF** all non-key attributes fully depend on key
  avoids much redundancy still has problems with some fds
- **3NF** avoids redundancy related to transitive dependancies between non prime attributes and keys still has redundancy problem
- **BCNF** has no redundancy, but some fds do not hold anymore

In practice, BCNF and 3NF are the most important for relational design.
We focus on these two (details of others may be found in any database textbook)

Normalisation Process

The normalisation process:

1. check whether a relation schema is in a particular normal form (xNF)
2. if not, partition into sub-relations where each is closer to xNF
   - The “reassembly" of sub-relations produces the original relation (*lossless join property*).
3. repeat until the desired xNF is achieved over all sub-relations

Decomposition and non-additive Joins.

Consider the relation:
Accounts (customer, accNo, balance, bank, address, assets)

- R1 = Accounts[accNo, balance, customer]
- R2 = Accounts[customer, bank, address, assets]
- R1 * R2 $\neq$ Accounts

*R1 * R2 has additional spurious tuples*
Such decomposition is called lossy decomposition

The value of R1 * R2 is:

customer	accNo	balance	bank	address	assets
1313131	101	600	Credit Cooperatif	Paris	9000000
1313131	102	400	Credit Cooperatif	Paris	9000000
1313131	101	100	Credit Cooperatif	Paris	9000000
1313131	101	500	La Nef	Paris	400000
1313131	101	100	La Nef	Paris	400000
1313131	102	400	La Nef	Paris	400000
1111111	201	500	Credit Cooperatif	Paris	9000000
1111111	204	900	Credit Cooperatif	Paris	9000000

Decomposition with non additive join property

A decomposition D = {R1 (X, Y), R2 (X, Z)} of R (X, Y, Z) has the non-additive join property $\Longleftrightarrow$

- X $\longrightarrow$ Y holds in R
- or X $\longrightarrow$ Z holds in R

In DB textbooks, this property is also called lossless join property

Relational decomposition into BCNF relations with lossless join property

Input : relation R + FDs for R
Output : decomposition of R into BCNF relations with "lossless join"

- *Step 1:* Compute keys for R
- *Step 2:* Repeat until all relations are in BCNF {
  - Choose a relation Q (X, Y, Z) that is not in BCNF
  - Find a FD X $\longrightarrow$ Y in Q that *violates* BCNF
  - Replace Q by *Q1 (X, Y)* and *Q2 (X, Z)*

  }

Warning : different order when choosing the FDs may result in different decompositions.

BCNF decomposition example

consider Tutors (<u>student, course</u>, tutor) ({tutor,student} is another key). Functional dependencies which hold in Tutors are:

- student, course $\longrightarrow$ tutor
- tutor $\longrightarrow$ course
- $\{\text{student, course}\}^+ = \{\text{student, course, tutor}\}$
- $\{\text{tutor}\}^+ = \{\text{course, tutor}\}$
- $\{\text{tutor, student}\}^+ = \{\text{student, course, tutor}\}$

The decomposition of Tutors into BCNF relations with lossless property is:

- Step 1: {student, course} and {student, tutor} are keys of R
- Step 2: Tutors is not in BCNF
  - tutor $\longrightarrow$ course violates the BCNF, and tutor itself is not a key.
  - Tutors is replaced by R1 (<u>tutor</u>, course) and R2 (<u>tutor, student</u>)
  - Both are in BCNF

Unfornately, the FD student, course $\longrightarrow$ tutor is not held any more in any of these relations.

Exercice (cont.)

Décomposer la relation EMPLOYES pour qu'elle soit en BCNF

EMPLOYES (NumE, Nom, Salaire, Département, Bâtiment)

NumE	Nom	Salaire	Dept.	Bâtiment
1	Dupont	2600	SI	IMAG
2	Martin	2400	BD	IMAG
3	Bernard	2100	BD	IMAG
4	Petit	2100	MATH	UFR IM2AG
5	Richard	2300	MATH	UFR IM2AG

NumE $\longrightarrow$ Nom, Salaire, Département

Département $\longrightarrow$ Bâtiment

Exercice (cont.)

Décomposer la relation EMPLOYES pour qu'elle soit en BCNF

EMPLOYES (NumE, Nom, Salaire, Département, Bâtiment)

NumE $\longrightarrow$ Nom, Salaire, Département

Département $\longrightarrow$ Bâtiment

Employes est separé en R1(NumE, Nom, Salaire, Département) et R2(Département, Bâtiment)

R1

NumE	Nom	Salaire	Dept.
1	Dupont	2600	SI
2	Martin	2400	BD
3	Bernard	2100	BD
4	Petit	2100	MATH
5	Richard	2300	MATH

R2

Dept.	Bâtiment
SI	IMAG
BD	IMAG
MATH	UFR IM2AG

36 Motivations

37 Design Anomalies

38 Functional Dependencies

39 Normalisation

40 Conclusion

Why is normalisation important?

- Functional dependencies and normalisation: nice tools to assess relation schemas
- Designing schema from scratch, how do we figure out
  - an attribute list to start with?
  - a set of functional dependencies?
- All these questions are answered when we design a UML class diagram.
- Denormalisation: for performance reasons relations may be left in a lower normal form.